

Data Structure(CSL202)

Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)



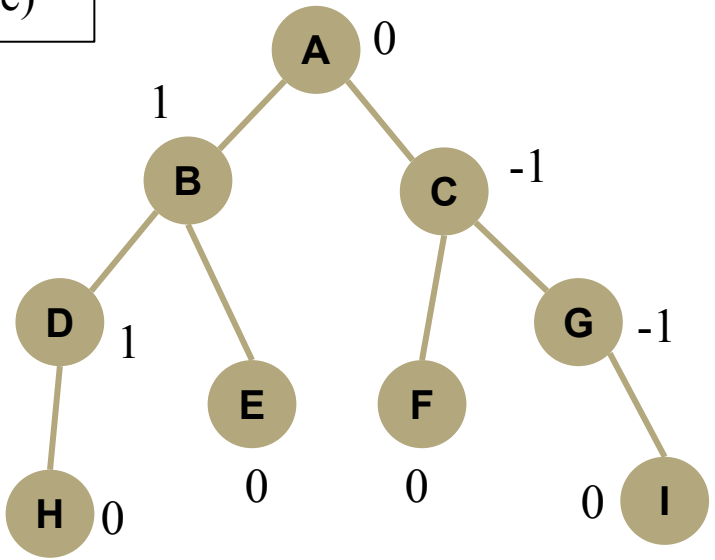
AVL (Adelson-Velskii and Landis)Tree

AVL (Adelson-Velskii and Landis)Tree

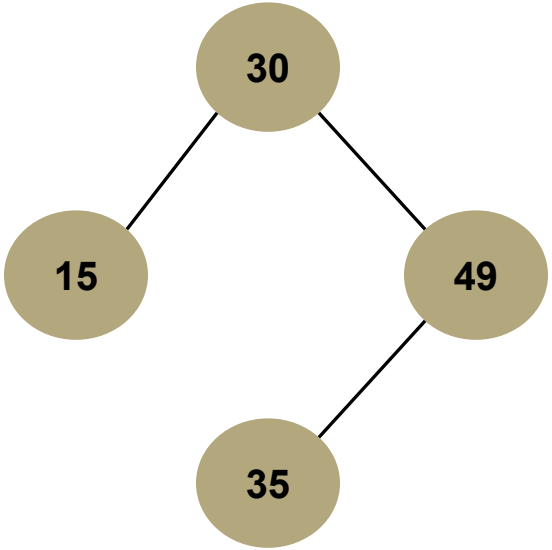
- Self-balancing BSTs.
- **For any node T:**

BalanceFactor(T) = height(left subtree) - height(right subtree)

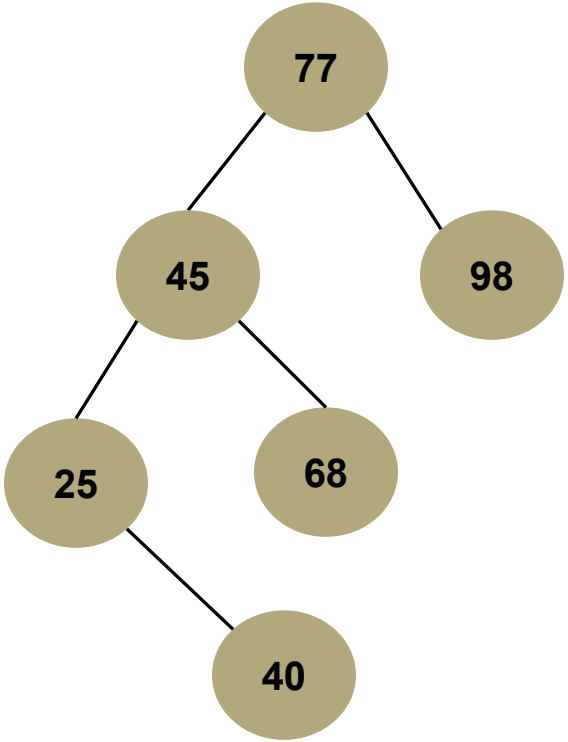
- Only -1, 0, +1 are allowed.
- Else, means the tree needs rebalancing.



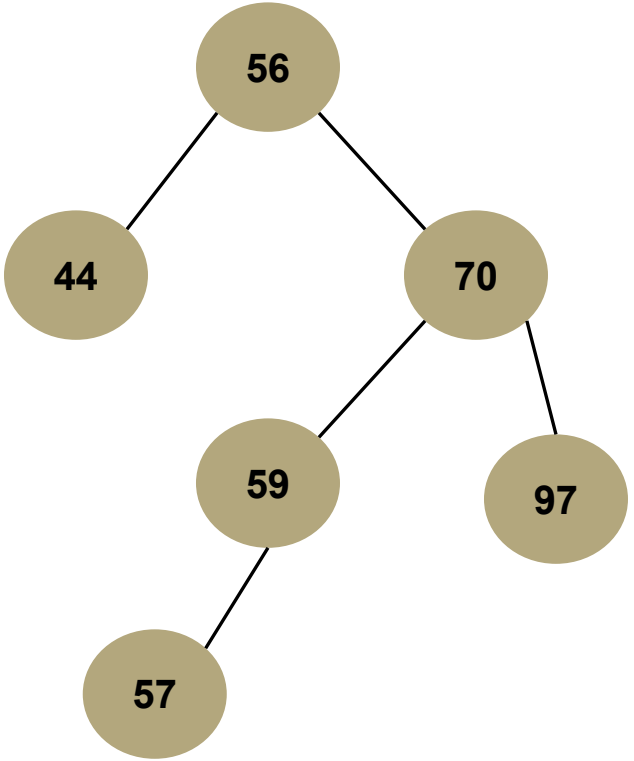
Identify AVL Tree



(a)

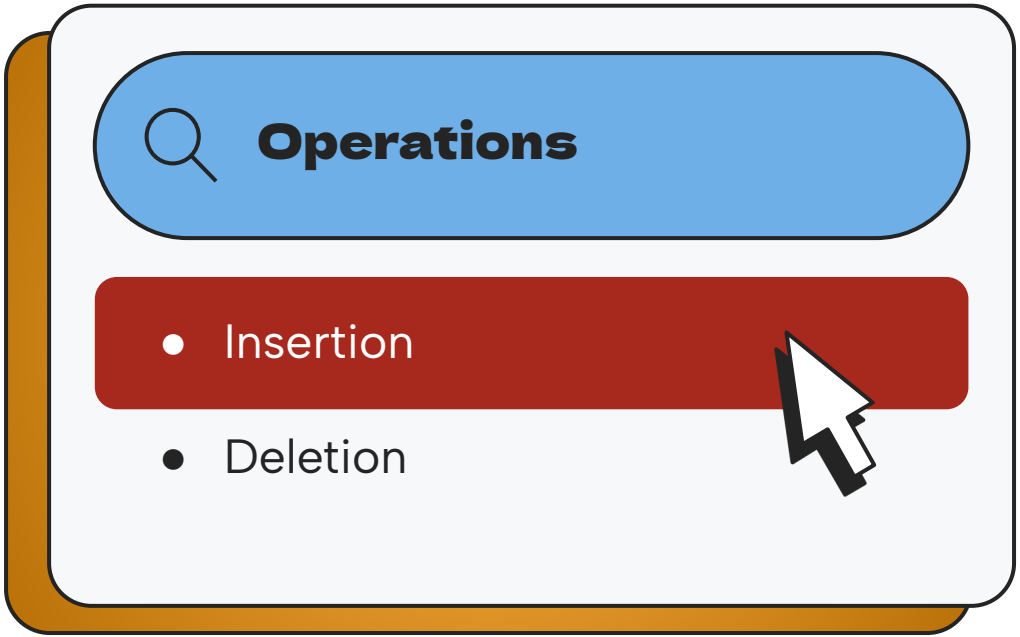


(b)



(c)

Operations in AVL Tree

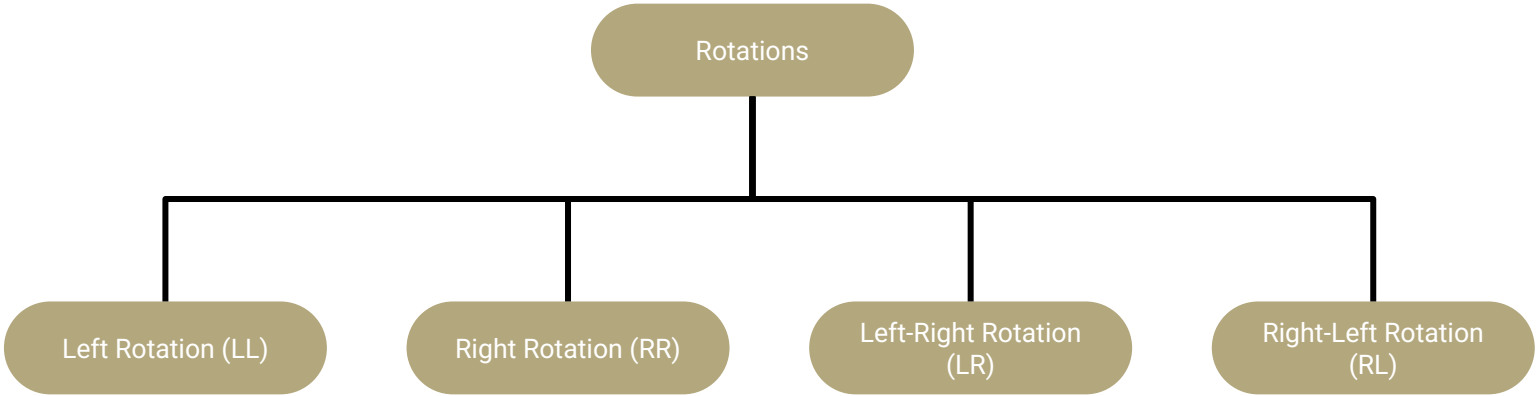


AVL Tree Insertion

- Step 1: Insert the new node like in a Binary Search Tree (BST).
- Step 2: Retrace the path back to the root, updating Balance Factors (BF):

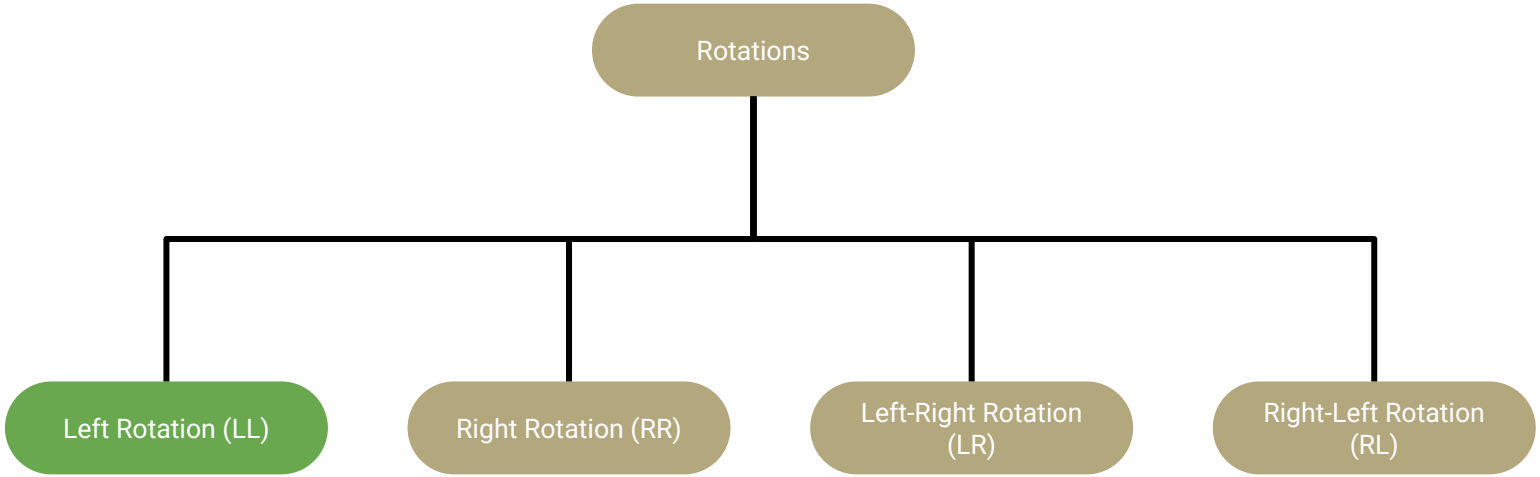
BF = height(left subtree) - height(right subtree)

- Step 3: If any node's BF becomes +2 or -2, apply rotations to balance:



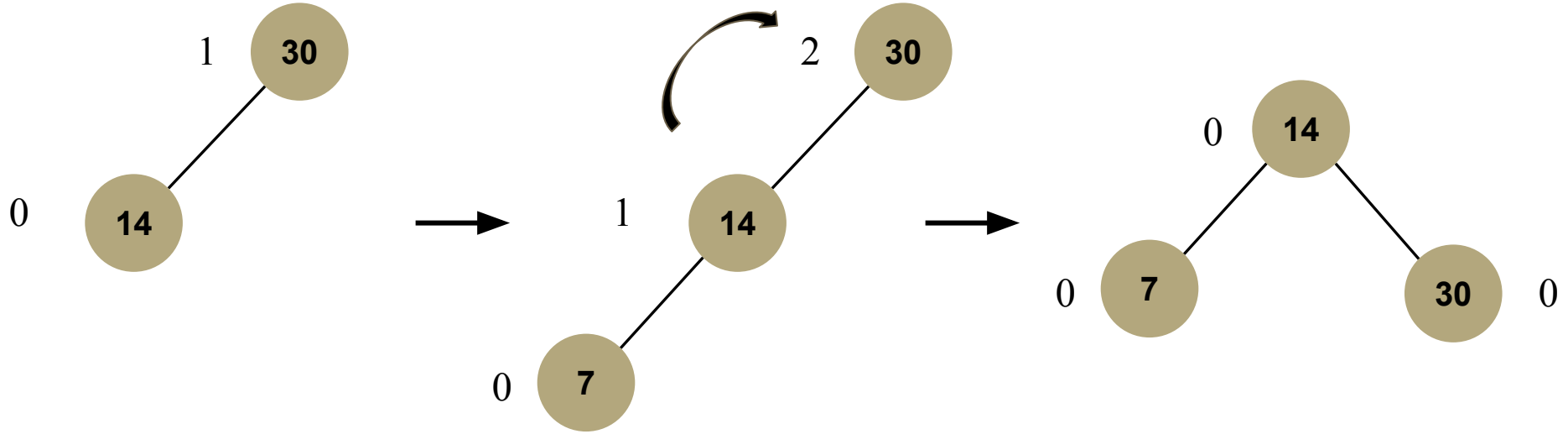
Rotations in AVL Tree

- Rotation is the process of moving the nodes to make tree balanced.



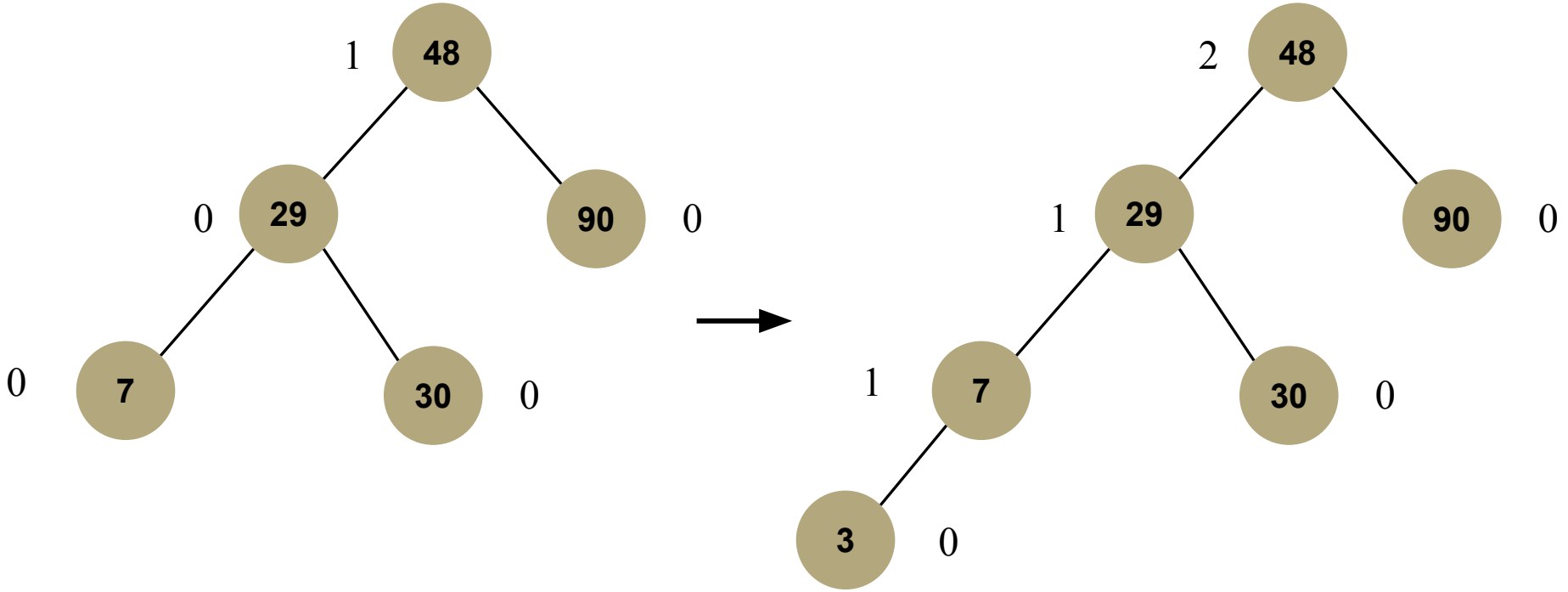
LL Rotation (due to insertion in Left subtree)

- An insertion into the left of the left subtree of node 14 causes imbalance.
- Needs a right rotation at A.



INSERT 7 UNBALANCED AVL BALANCED AVL

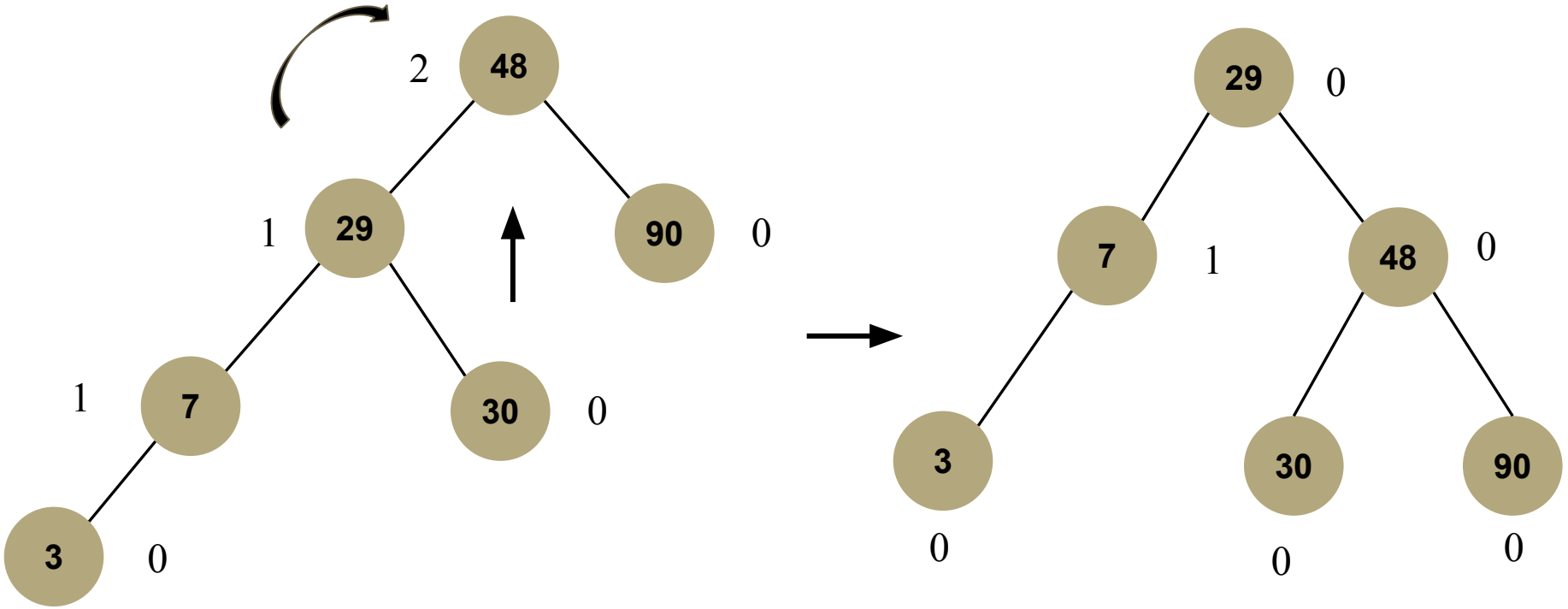
LL Rotation



INSERT 3

UNBALANCED AVL

LL Rotation

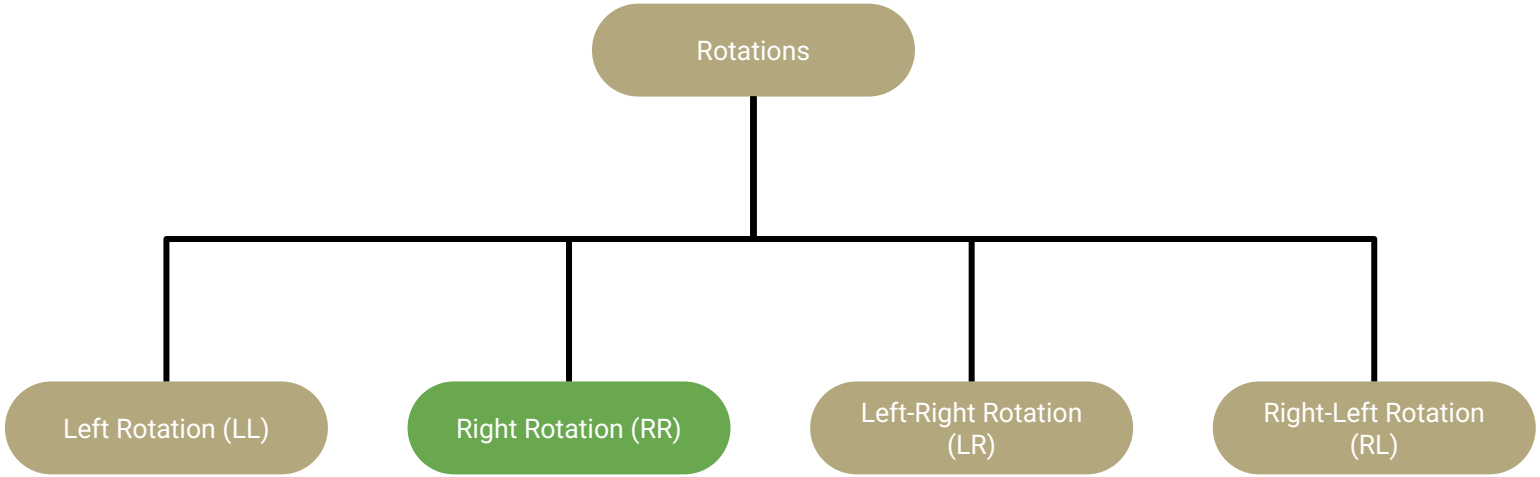


UNBALANCED AVL

BALANCED AVL

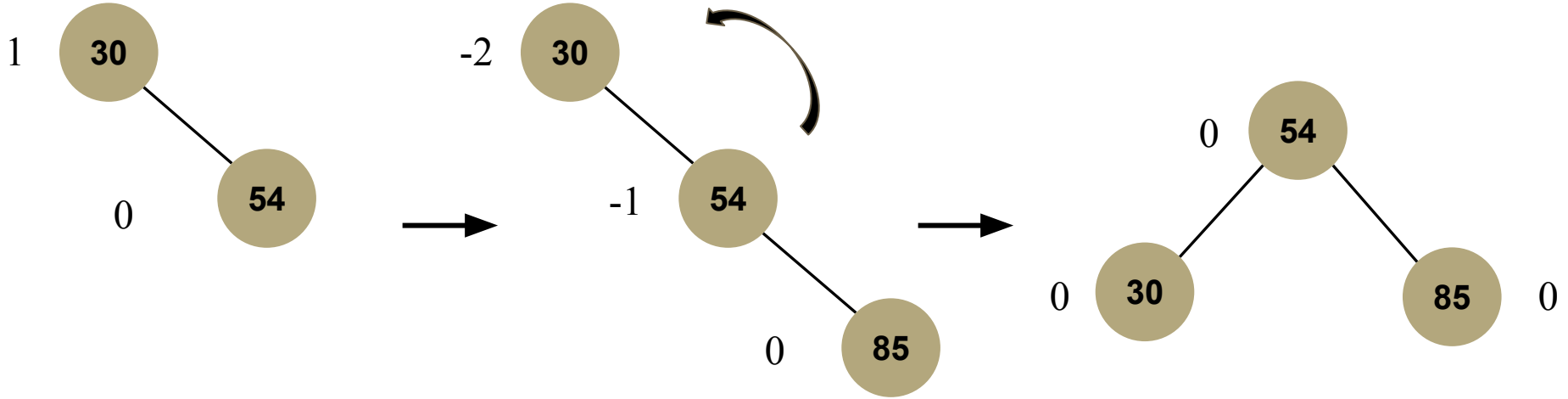
Rotations in AVL Tree

- Rotation is the process of moving the nodes to make tree balanced.



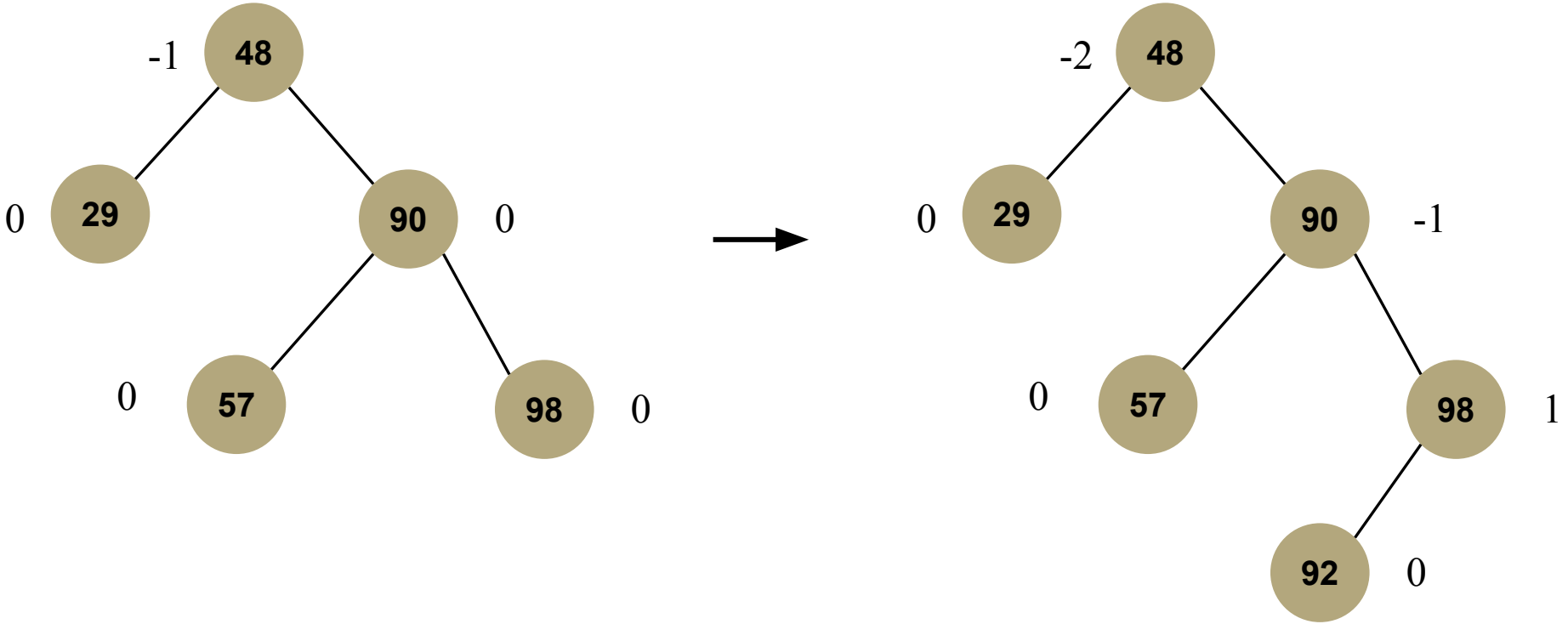
RR Rotation

- An insertion into the right of the right subtree of node A causes imbalance.
- Needs a left rotation at A.



INSERT 7 UNBALANCED AVL BALANCED AVL

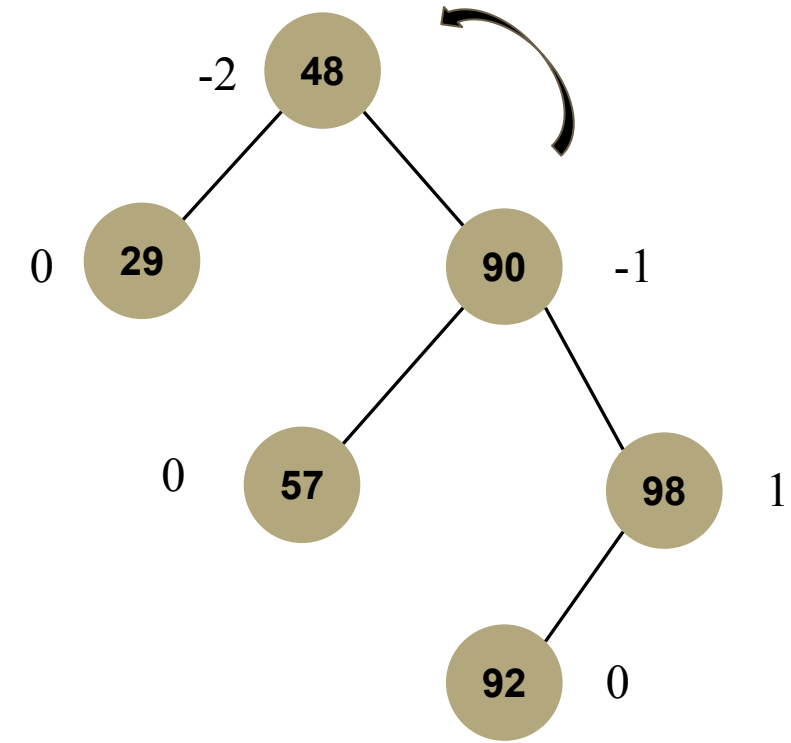
RR Rotation



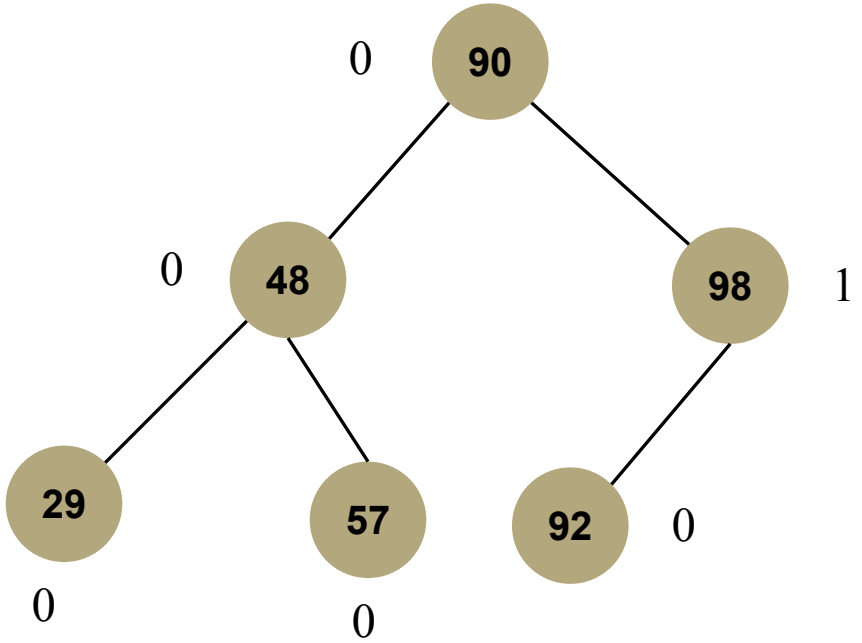
a. INSERT 92

b. UNBALANCED AVL

RR Rotation



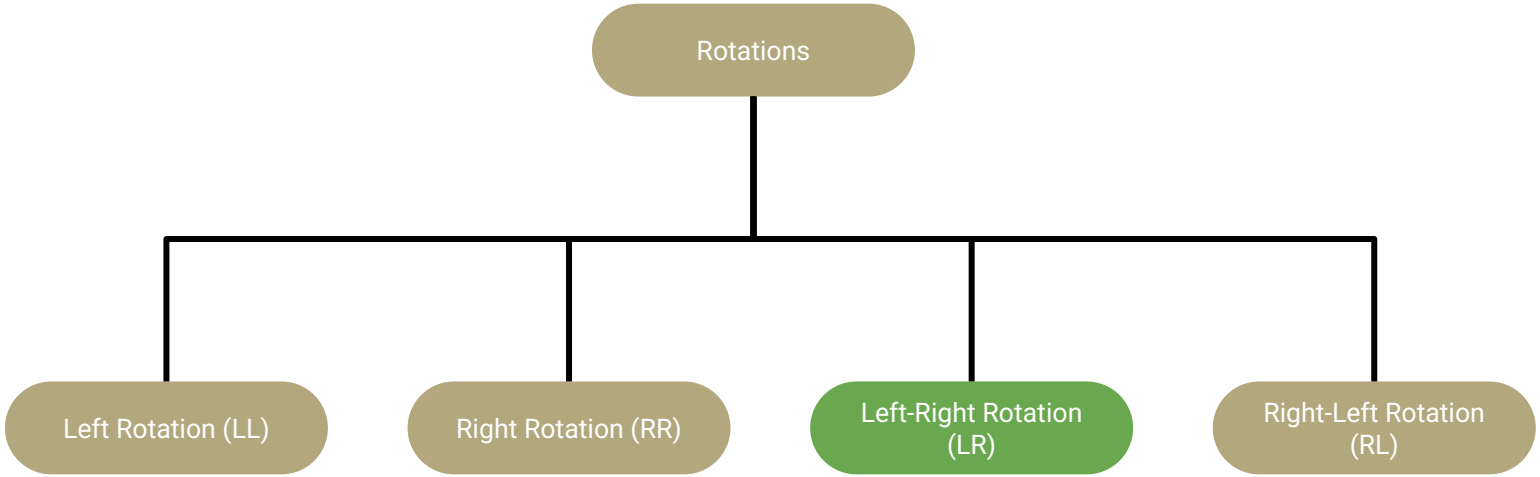
b. UNBALANCED AVL



c. BALANCED AVL

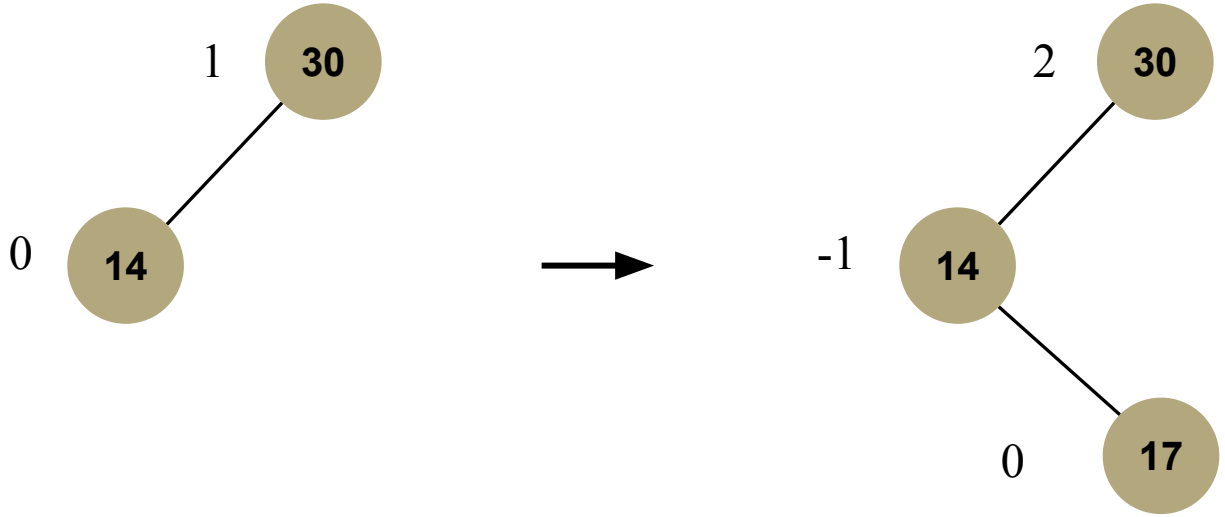
Rotations in AVL Tree

- Rotation is the process of moving the nodes to make tree balanced.



LR Rotation

- An insertion into the right of the left subtree of node A causes imbalance.
- First left rotation at B, then right rotation at A.

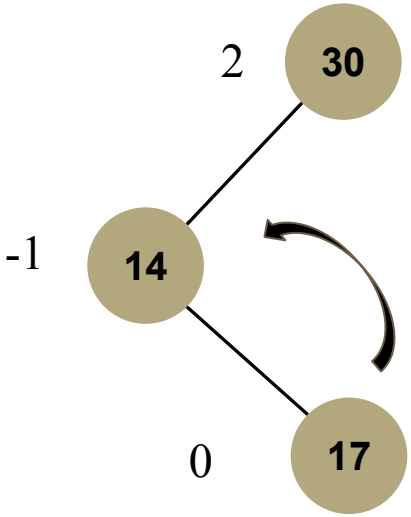


a. INSERT 17

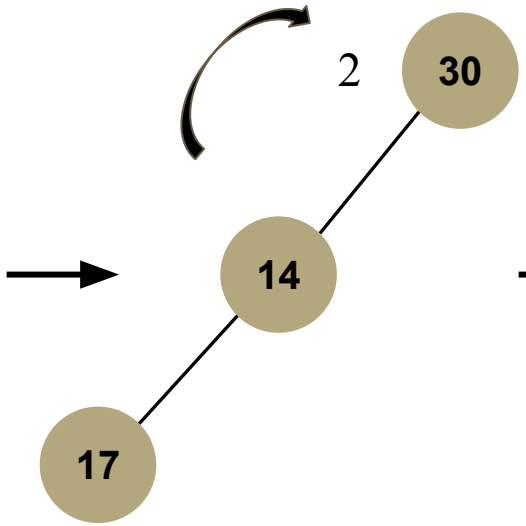
b. UNBALANCED AVL

LR Rotation

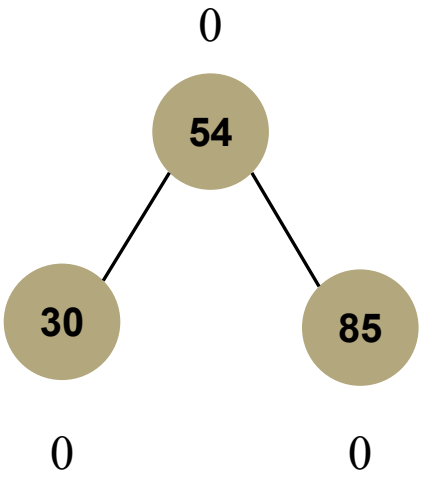
- An insertion into the right subtree of the left subtree of node A causes imbalance.
- First left rotation at B, then right rotation at A.



b. UNBALANCED AVL

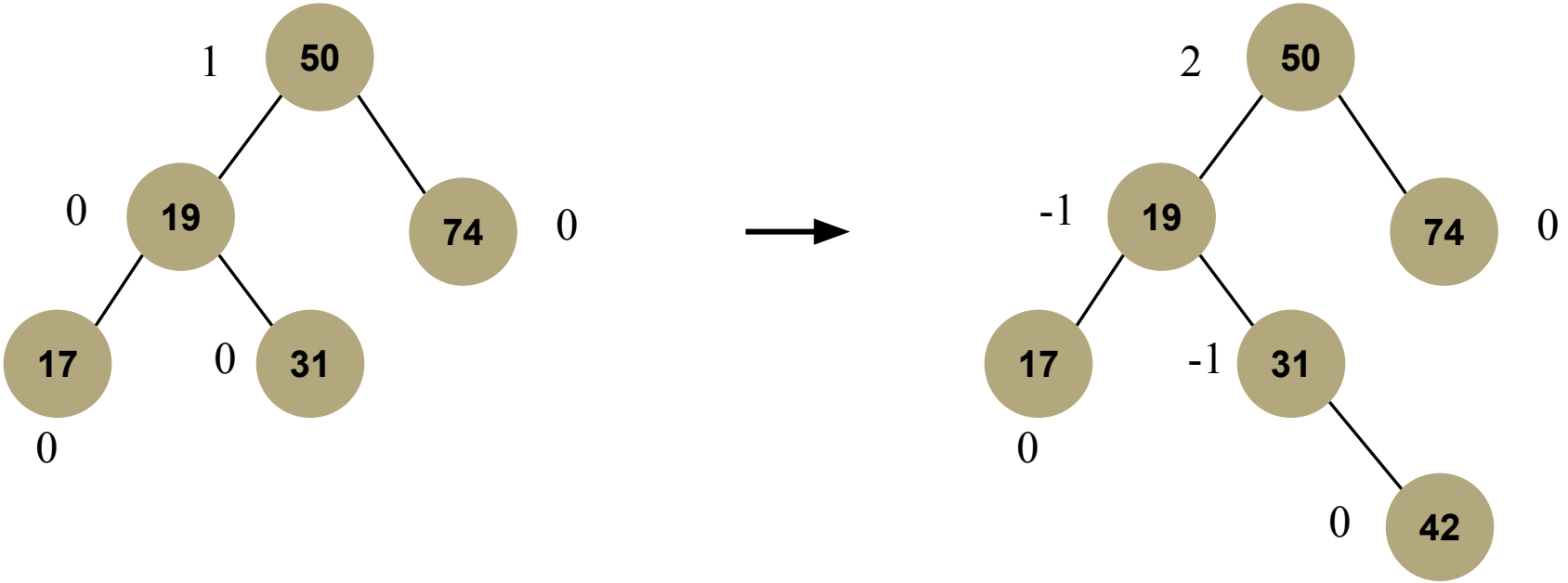


c. UNBALANCED AVL



d. BALANCED AVL

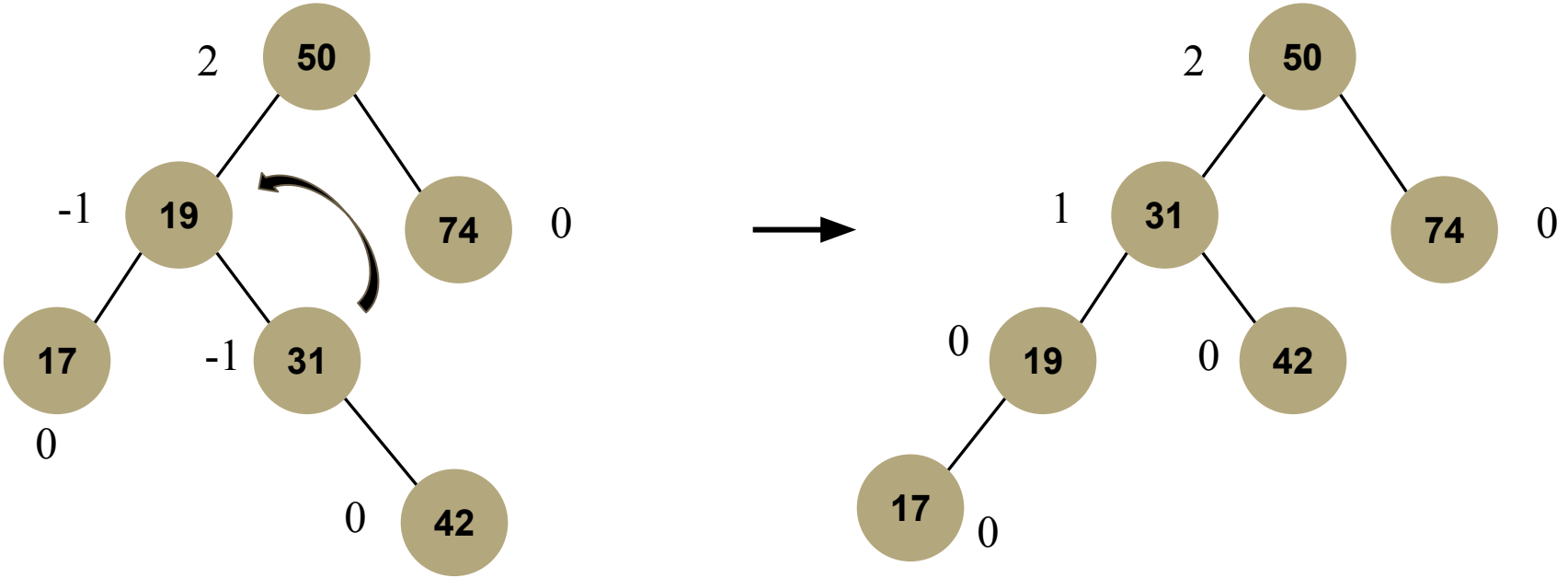
LR Rotation



a. INSERT 42

b. UNBALANCED AVL

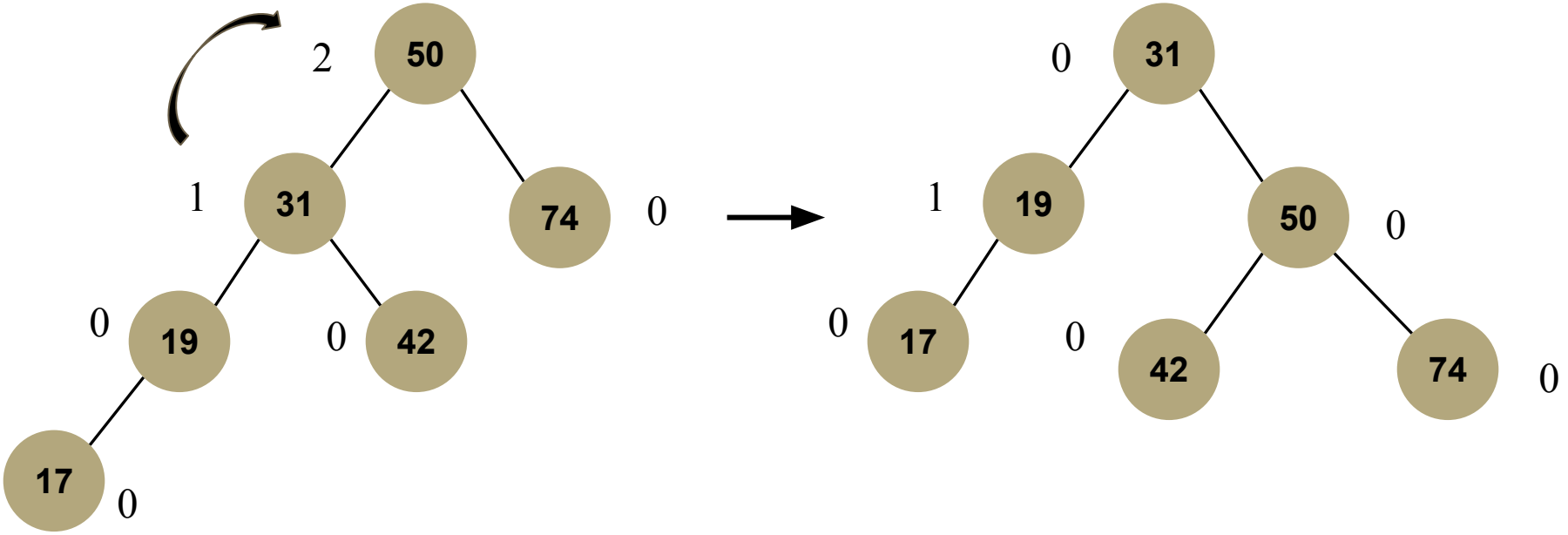
LR Rotation



b. UNBALANCED AVL

c. UNBALANCED AVL

LR Rotation

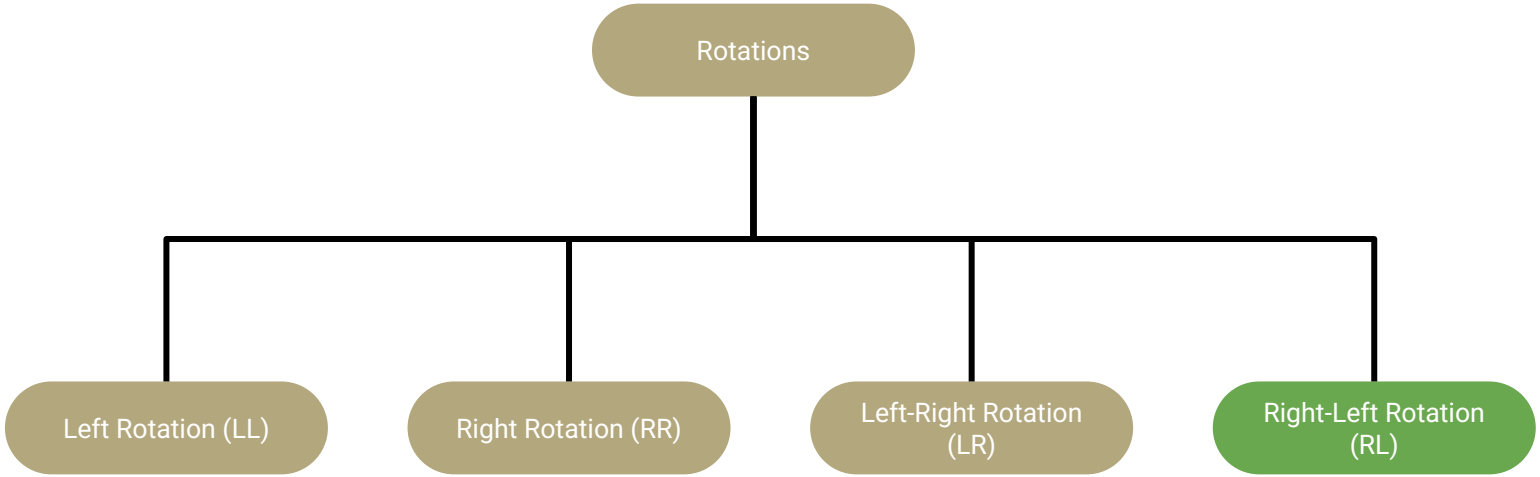


c. UNBALANCED AVL

d. BALANCED AVL

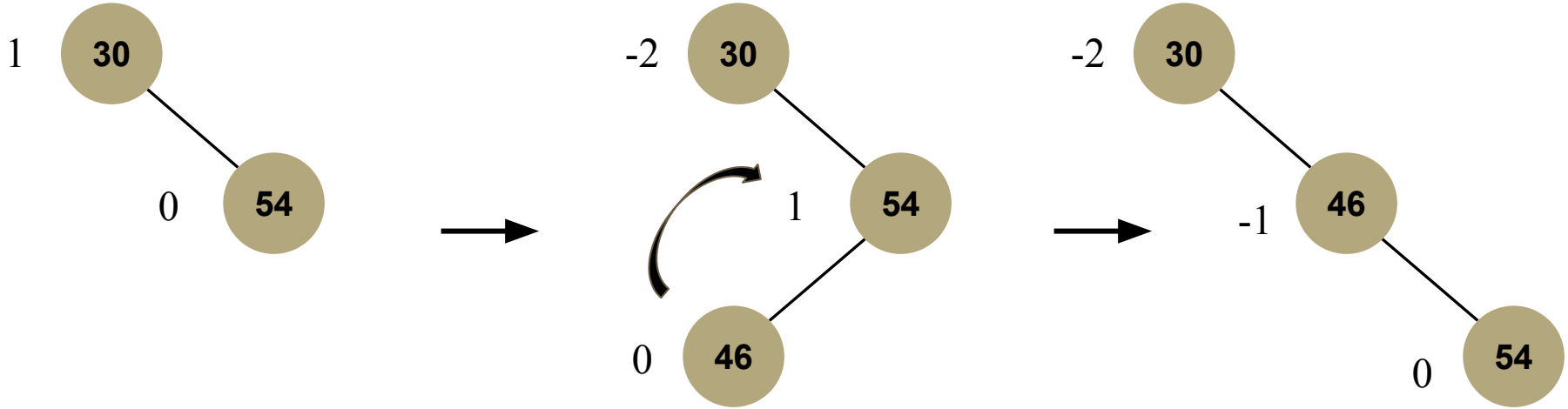
Rotations in AVL Tree

- Rotation is the process of moving the nodes to make tree balanced.



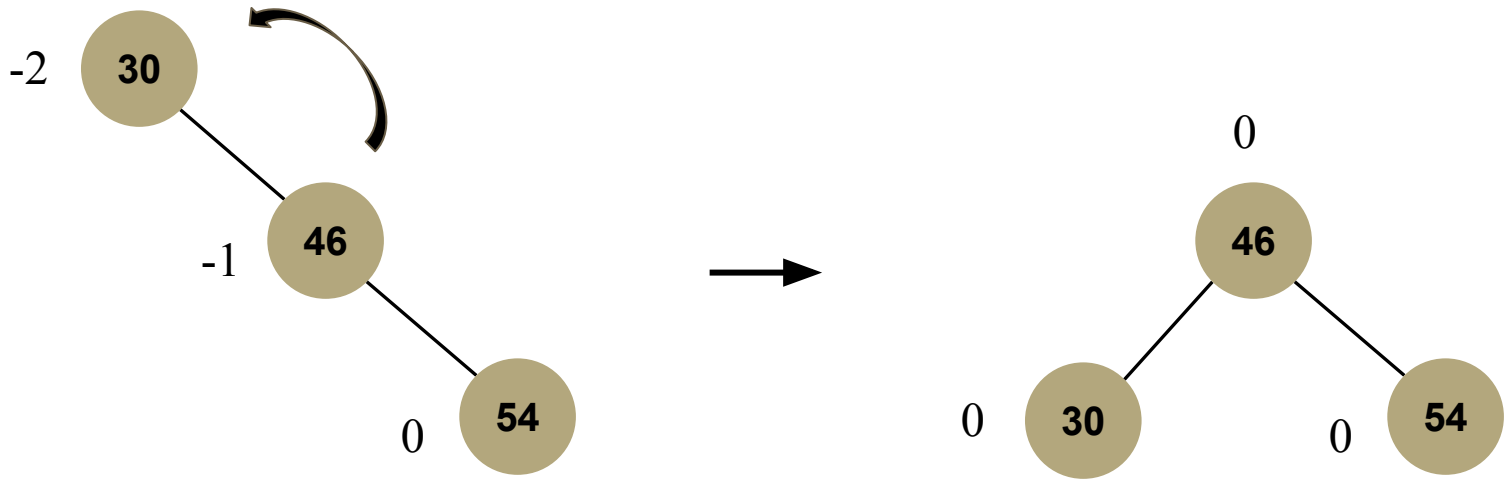
RL Rotation

- An insertion into the left subtree of the right subtree of node A causes imbalance.
- First right rotation at B, then left rotation at A.



a. INSERT 46 b. UNBALANCED AVL c. UNBALANCED AVL

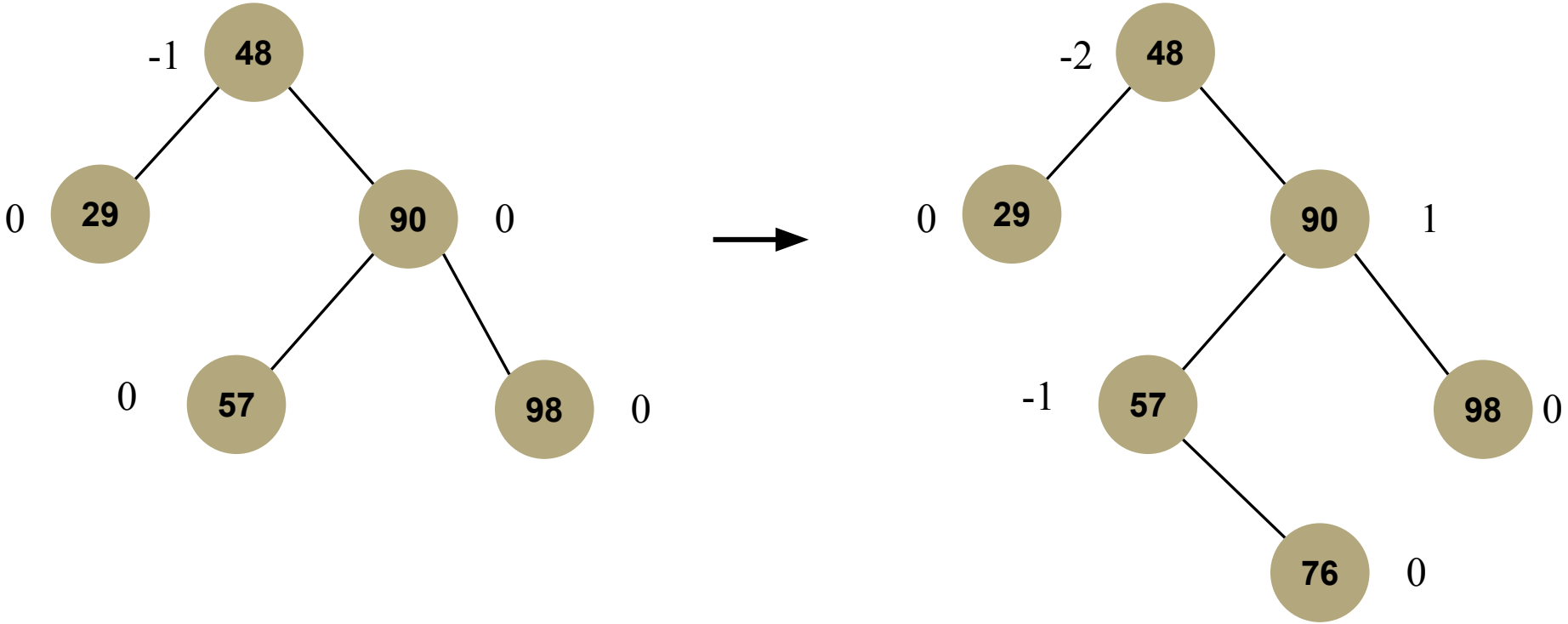
RL Rotation



c. UNBALANCED AVL

d. BALANCED AVL

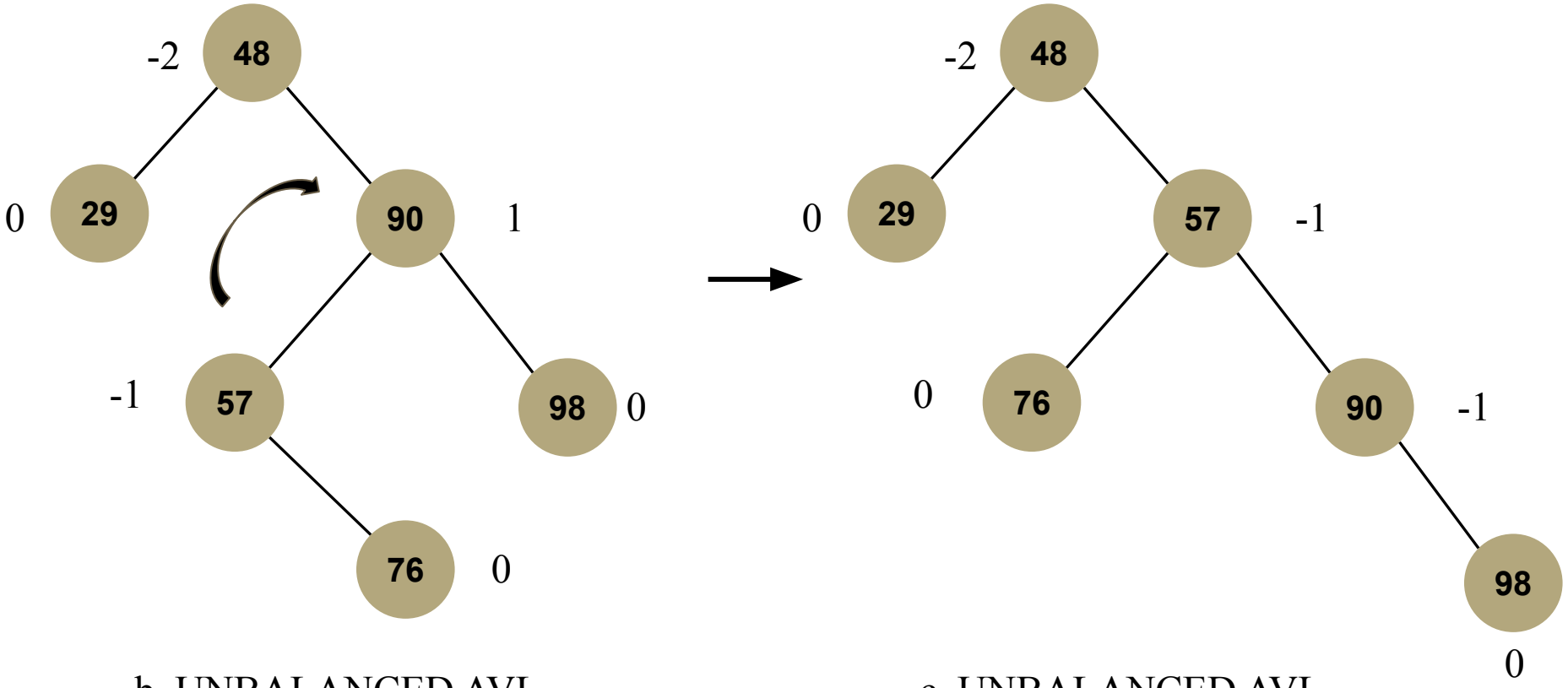
RL Rotation



a. INSERT 76

b. UNBALANCED AVL

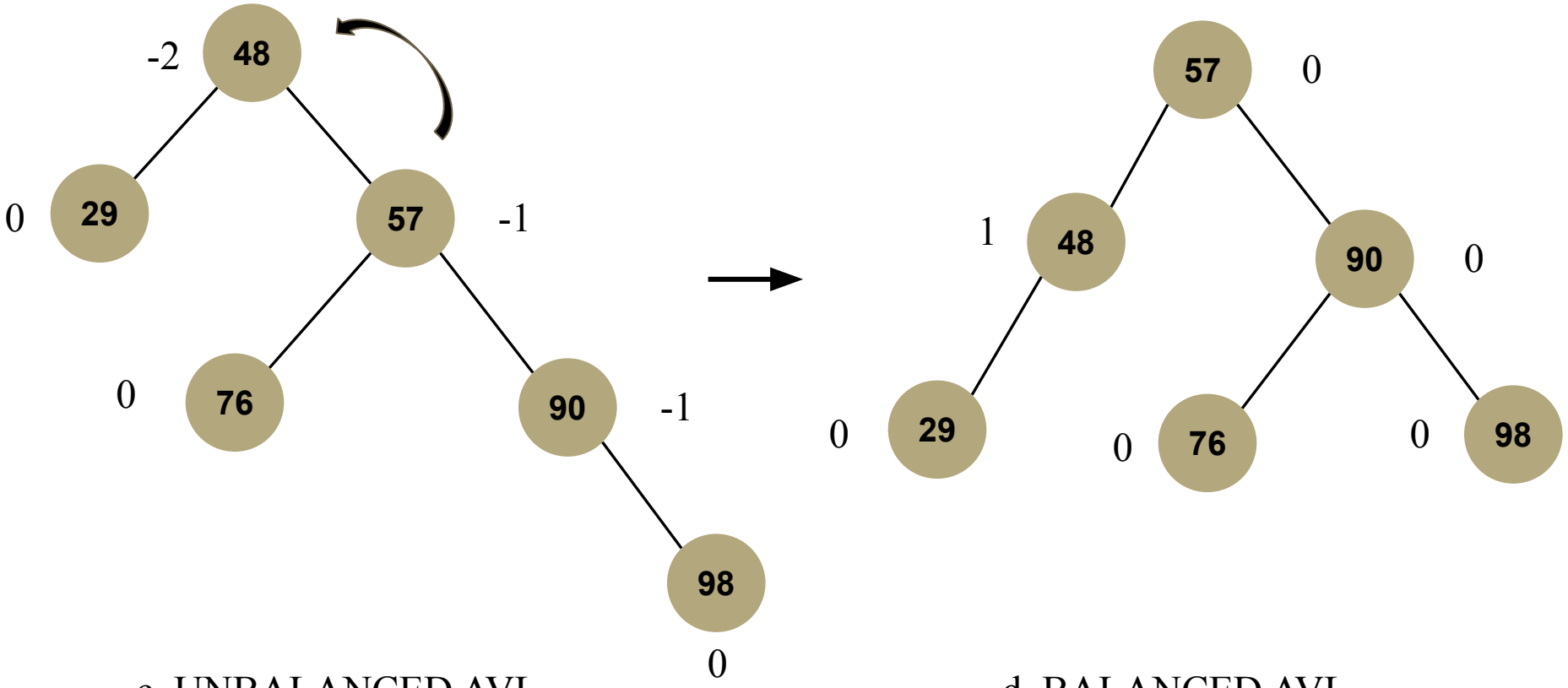
RL Rotation



b. UNBALANCED AVL

c. UNBALANCED AVL

RL Rotation



c. UNBALANCED AVL

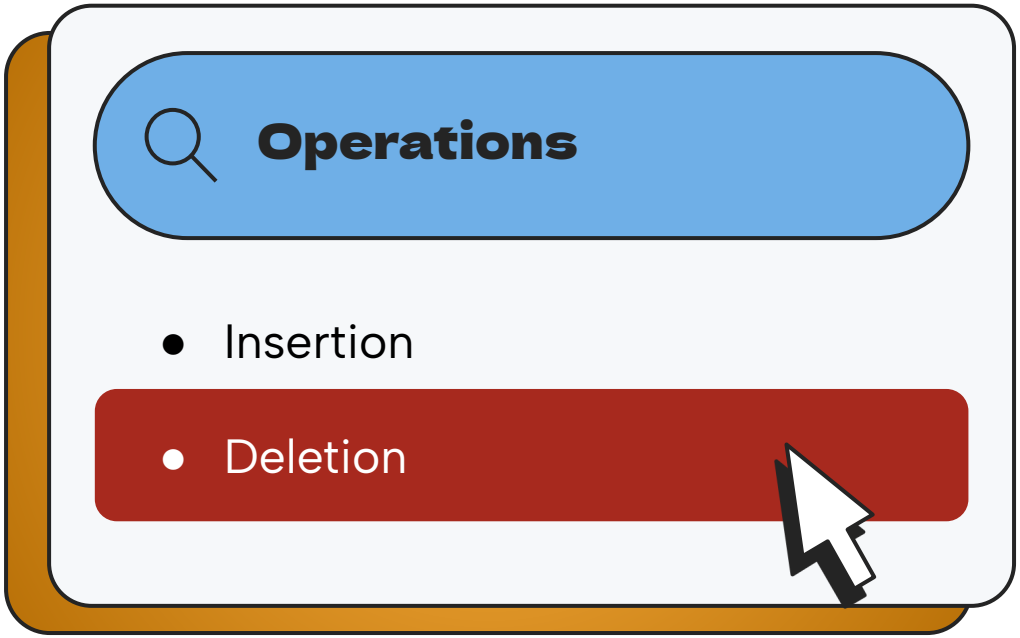
d. BALANCED AVL

AVL Tree Insertion Complexity

Since an AVL tree is always height-balanced, the height remains O(log n)

- **Best Case:** O(log n)
- **Average Case:** O(log n)
- **Worst Case:** O(log n)

Operations in AVL Tree

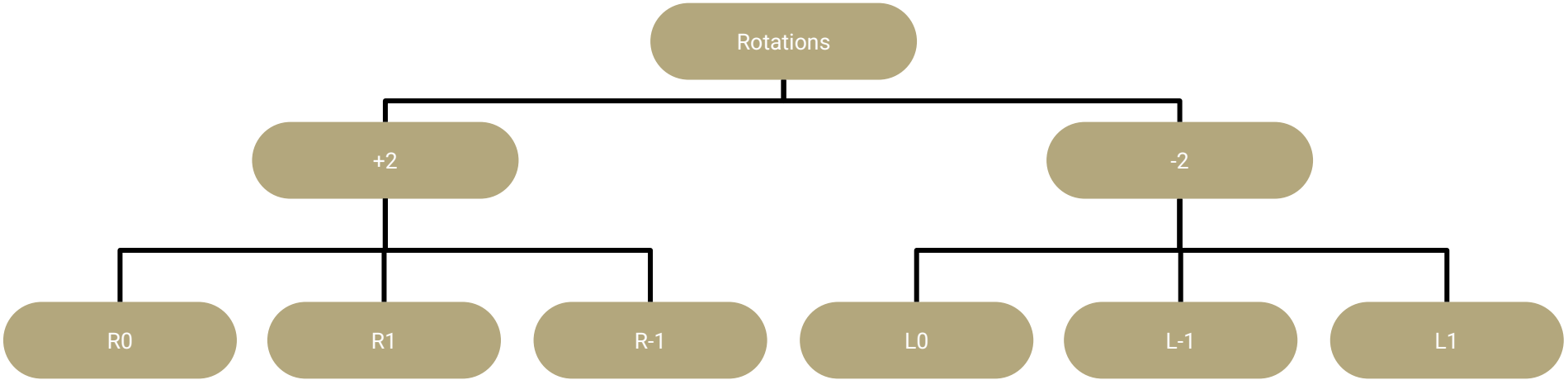


AVL Tree Deletion

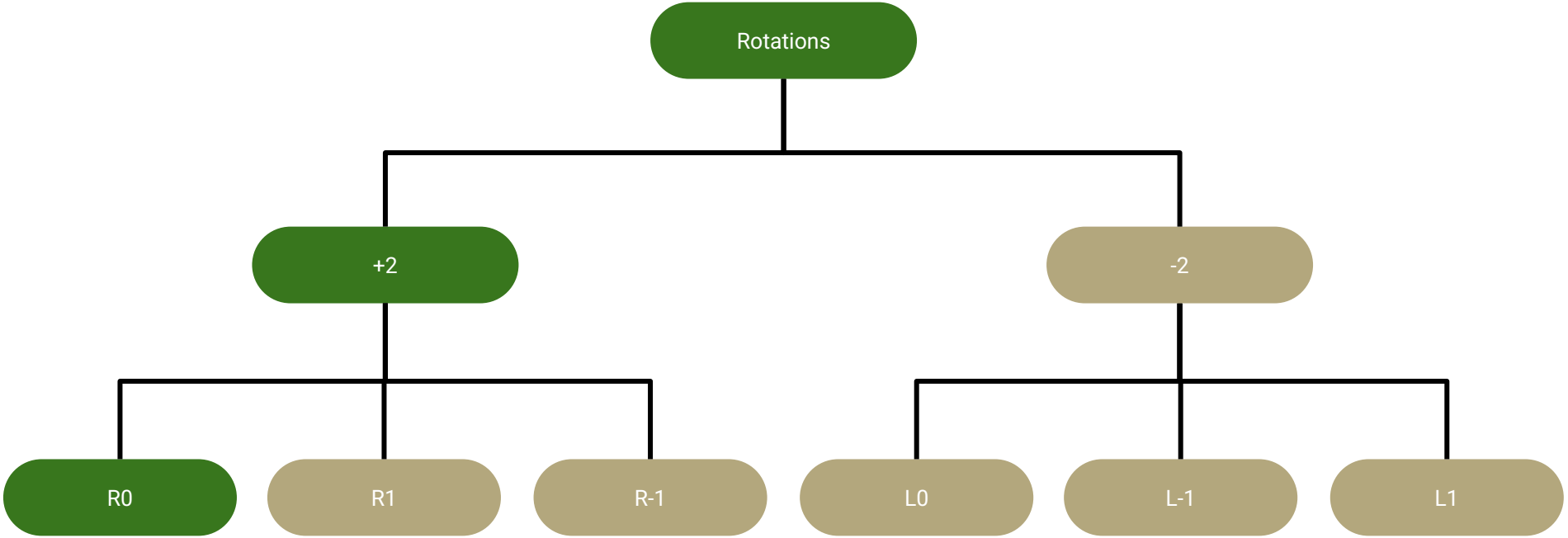
- Step 1: Delete the node like from a Binary Search Tree (BST).
- Step 2: Retrace the path back to the root, updating Balance Factors (BF):

BF = height(left subtree) - height(right subtree)

- Step 3: If any node's BF becomes +2 or -2, apply rotations to balance:

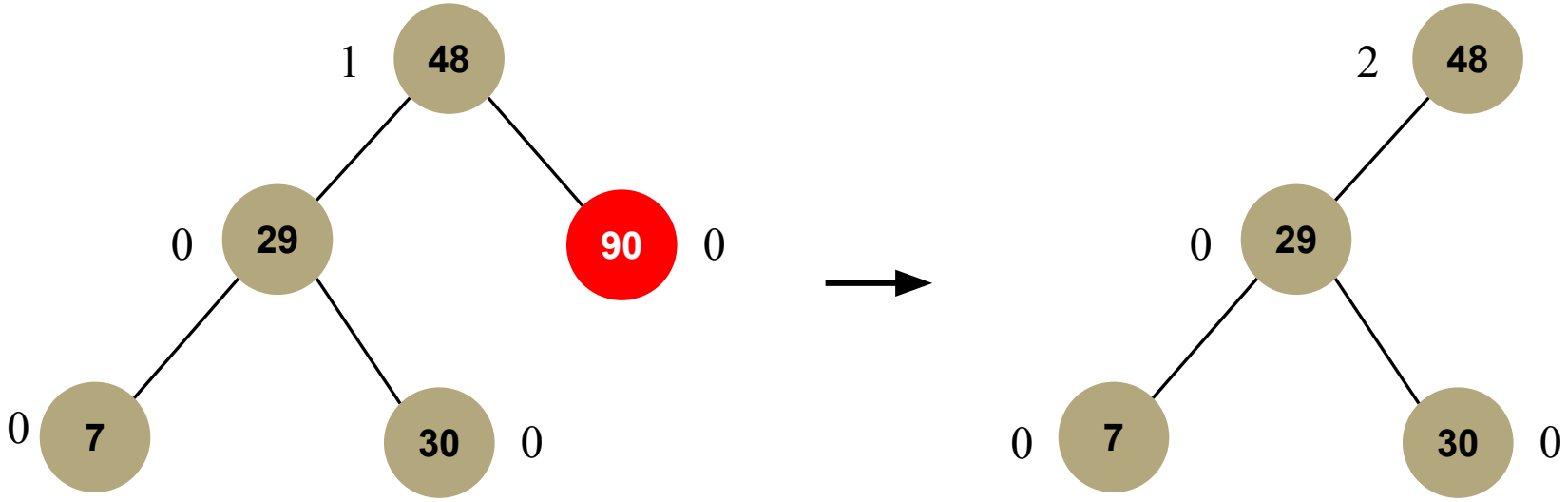


AVL Tree Deletion



R0 Rotation

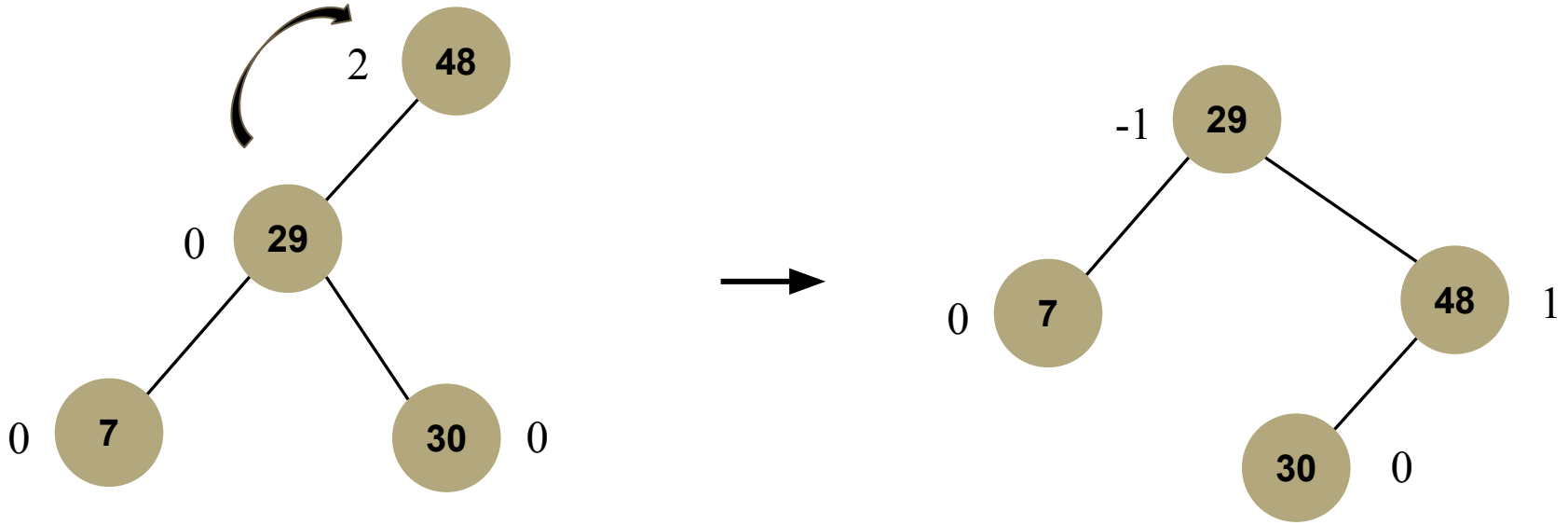
- Deletion occurred in the right subtree of A → BF(A) = +2 and (BF(B) = 0).
- Requires single right rotation around node A.



a. DELETE 90

b. UNBALANCED AVL

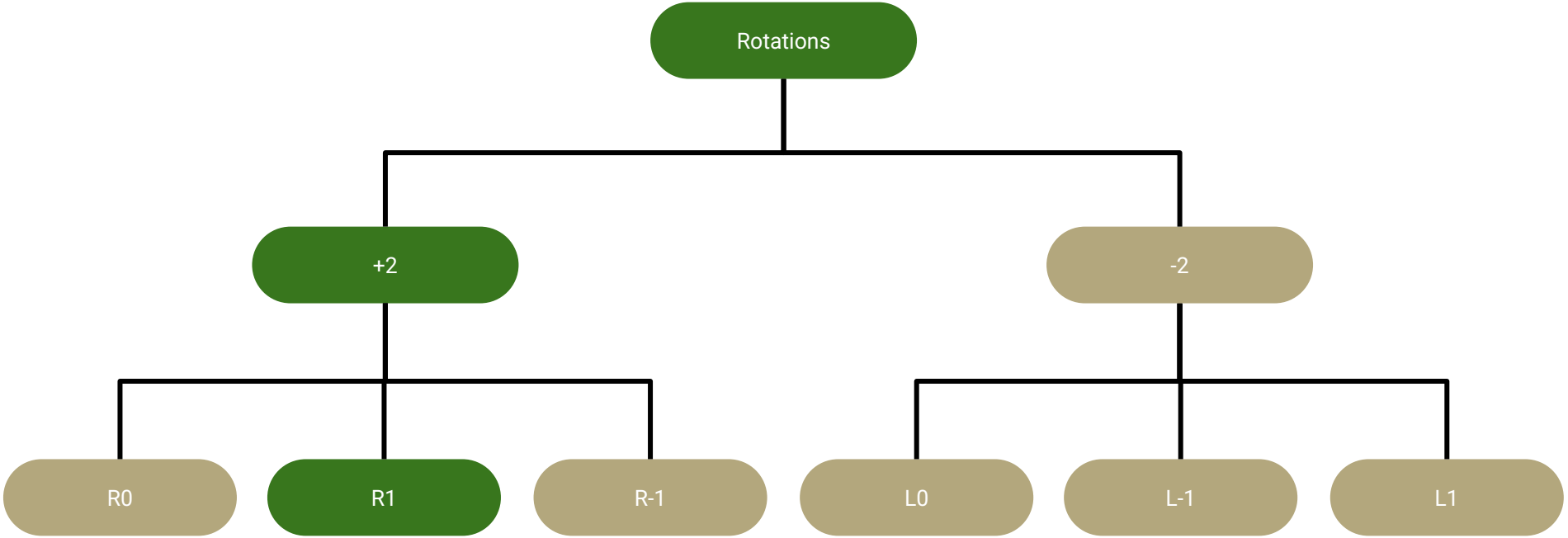
R0 Rotation



b. UNBALANCED AVL

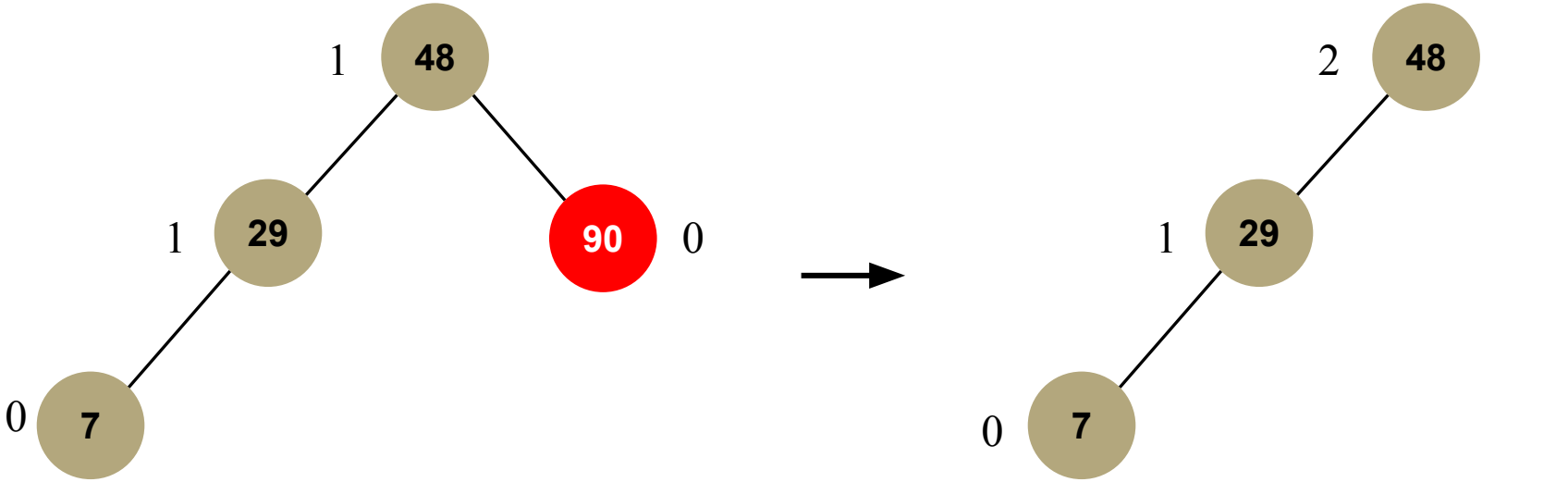
c. BALANCED AVL

AVL Tree Deletion



R1 Rotation

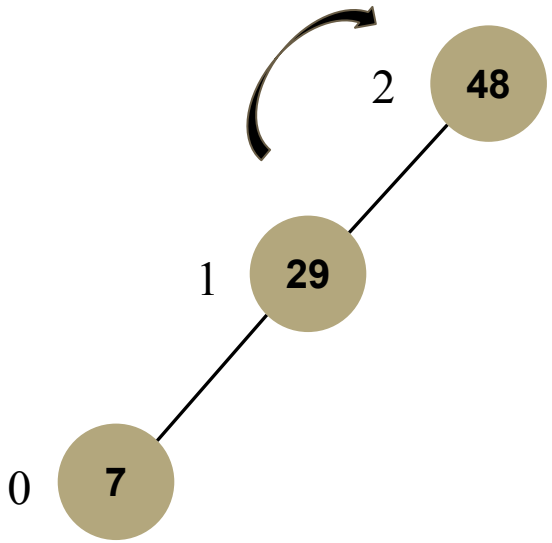
- Deletion occurred in the right subtree of A → BF(A) = +2 and (BF(B) = 1).
- Requires single right rotation around node A.



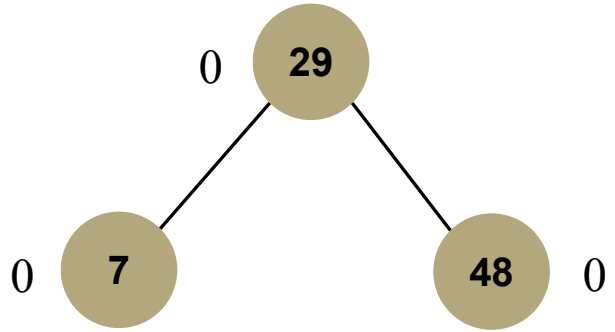
a. DELETE 90

b. UNBALANCED AVL

R1 Rotation

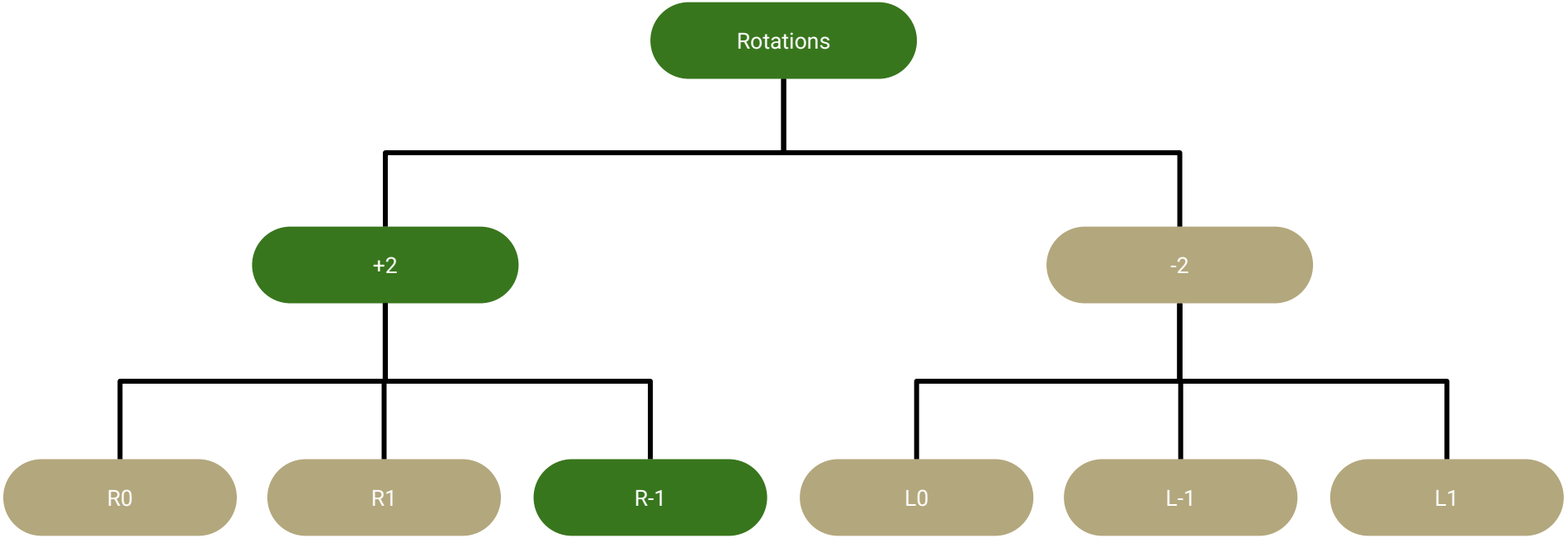


b. UNBALANCED AVL



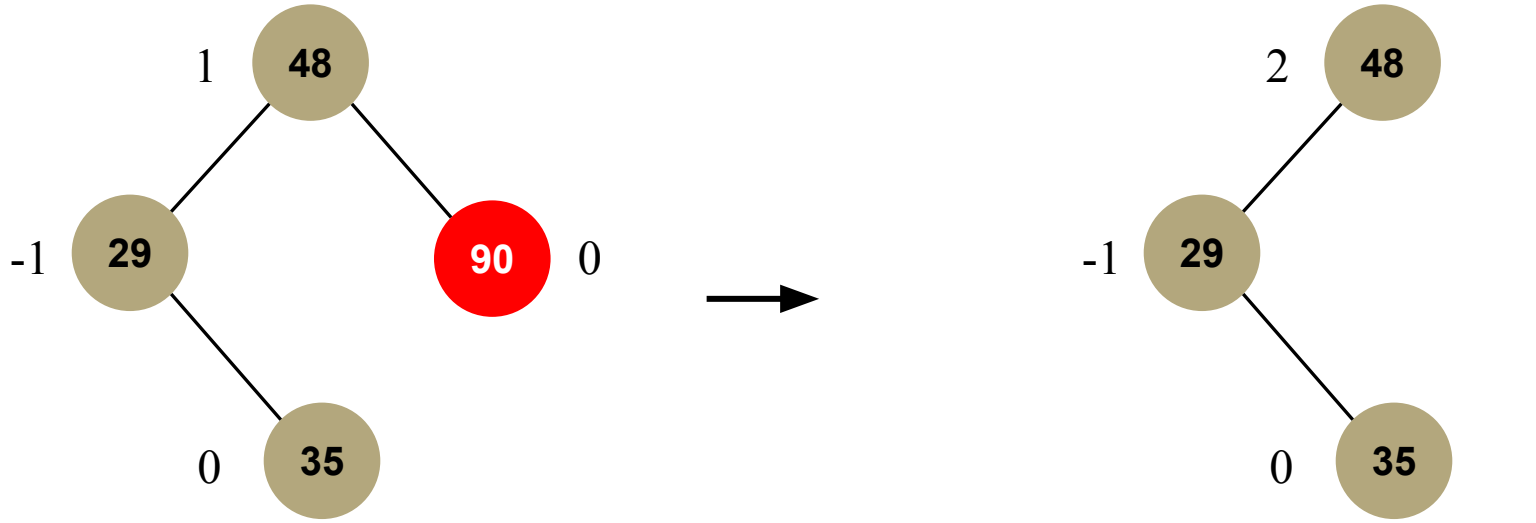
c. BALANCED AVL

AVL Tree Deletion



R-1 Rotation

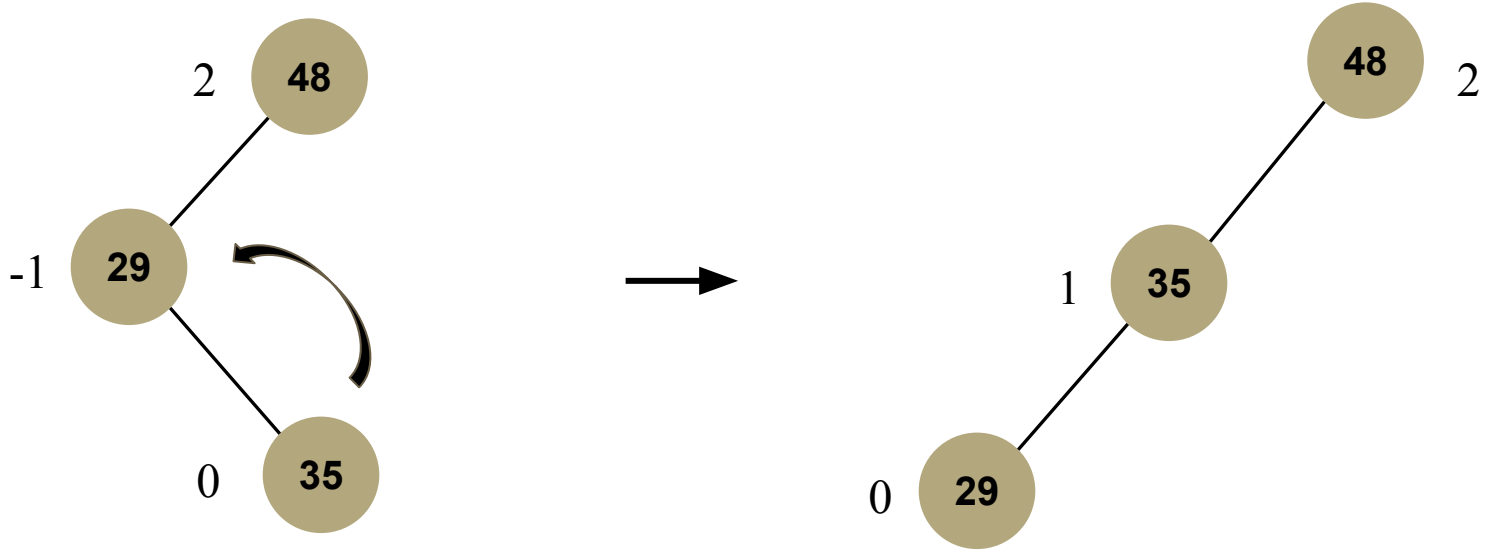
- Deletion occurred in the right subtree of A → BF(A) = +2 and (BF(B) = -1).
- Requires first left rotation on B, then right rotation on A.



a. DELETE 90

b. UNBALANCED AVL

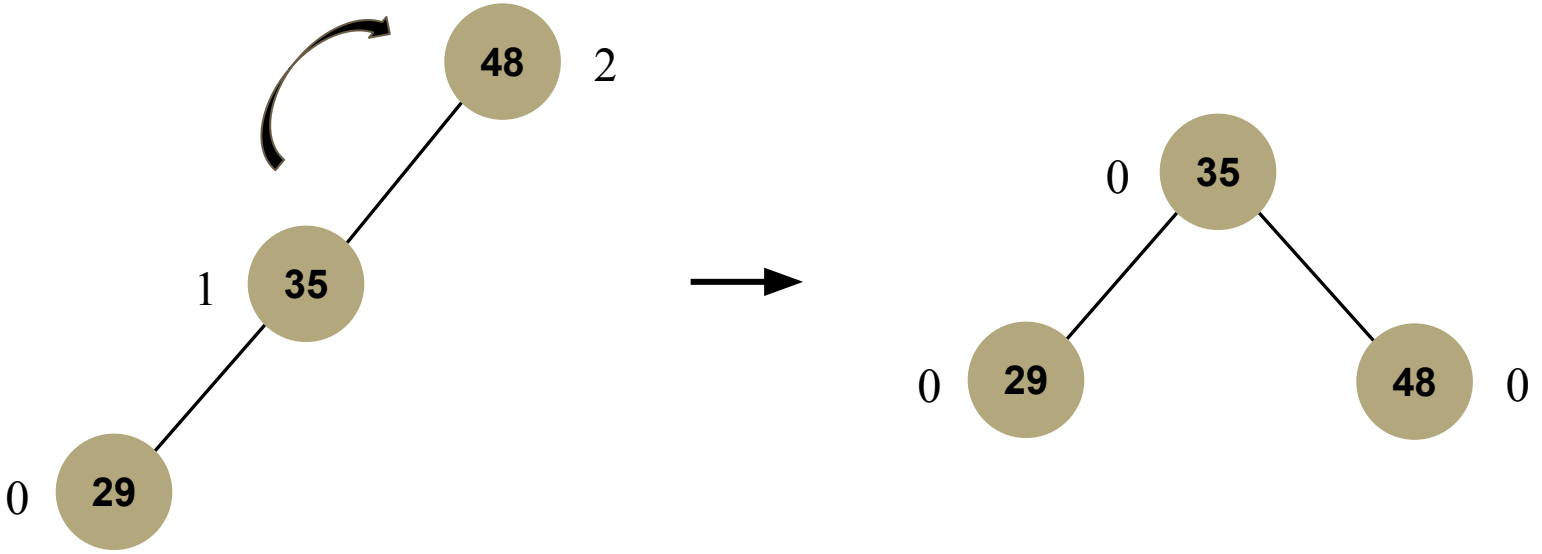
R-1 Rotation



b. UNBALANCED AVL

c. BALANCED AVL

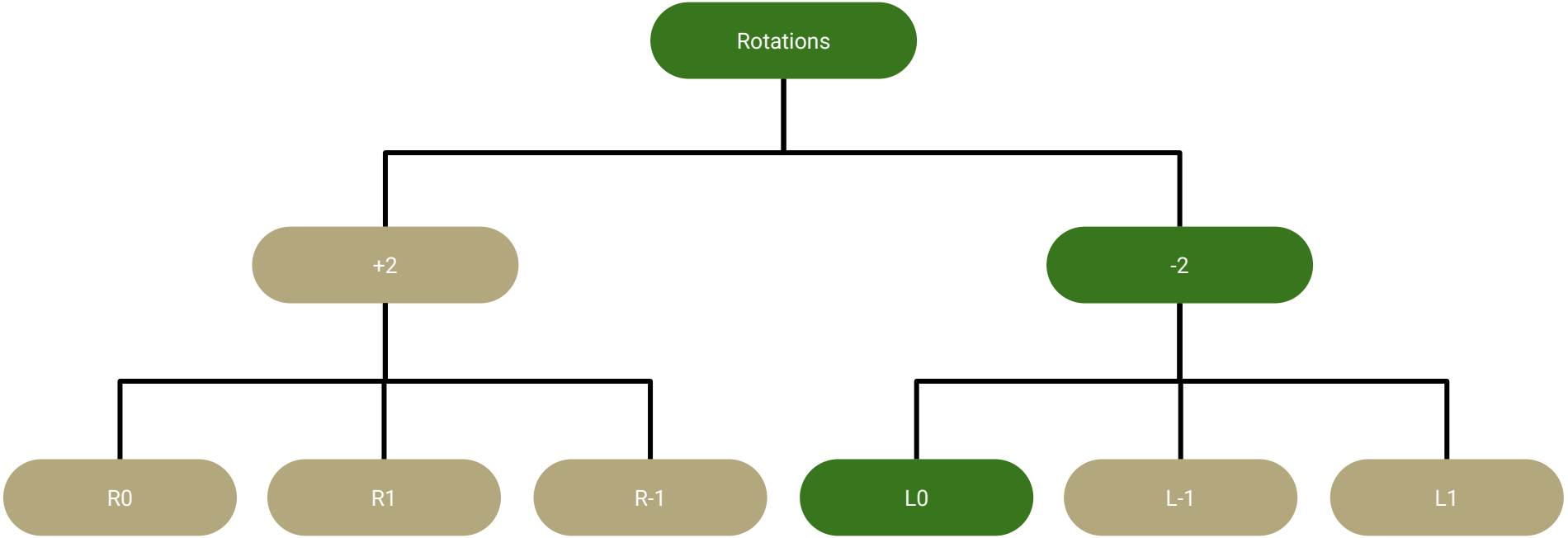
R-1 Rotation



c. UNBALANCED AVL

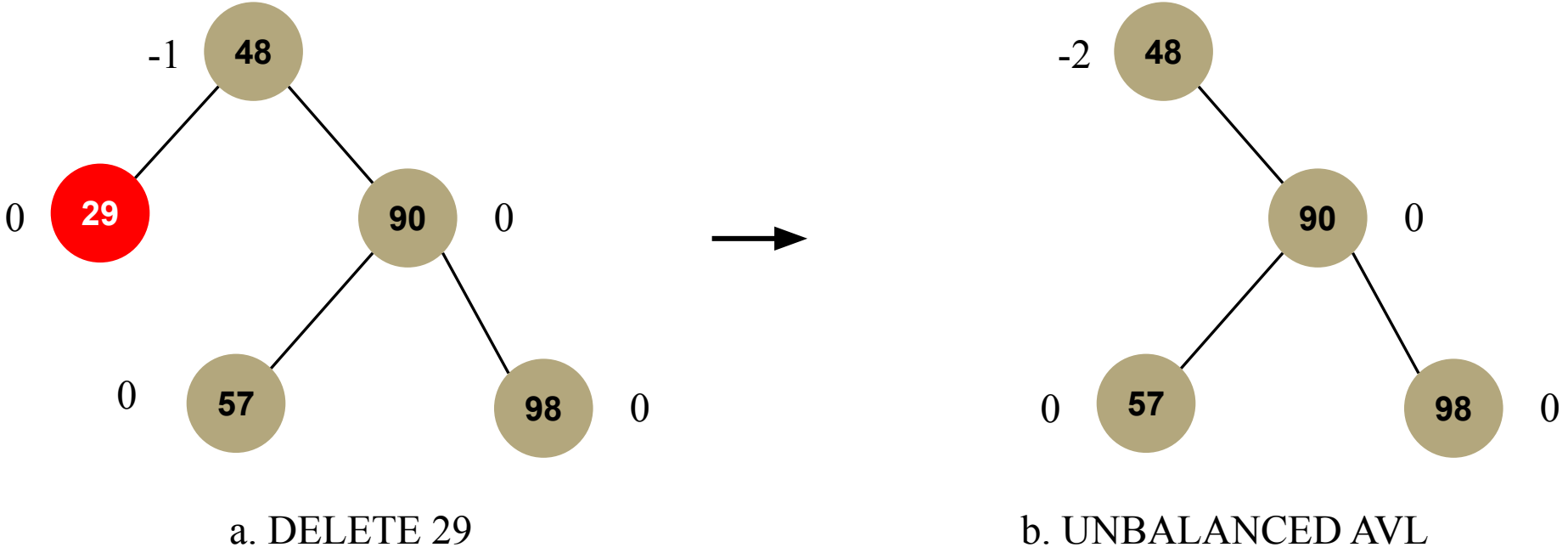
d. BALANCED AVL

AVL Tree Deletion



L0 Rotation

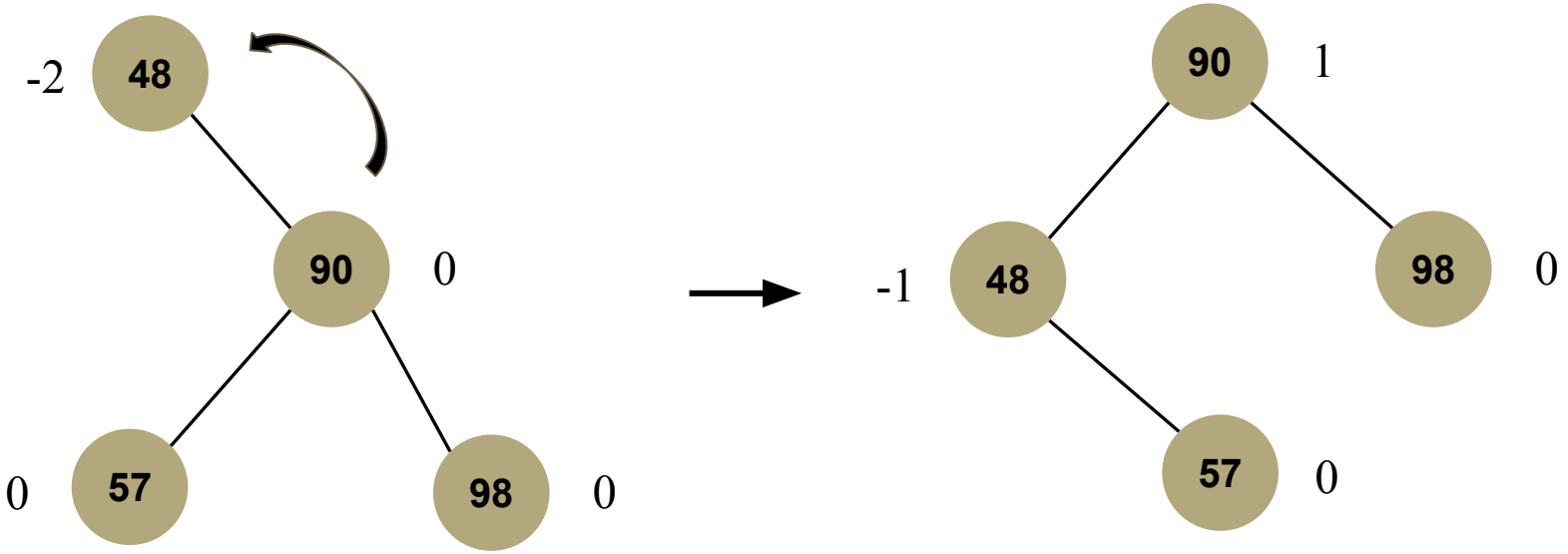
- Deletion occurred in the left subtree of A → BF(A) = -2 and (BF(B) = 0).
- Requires single left rotation around node A.



a. DELETE 29

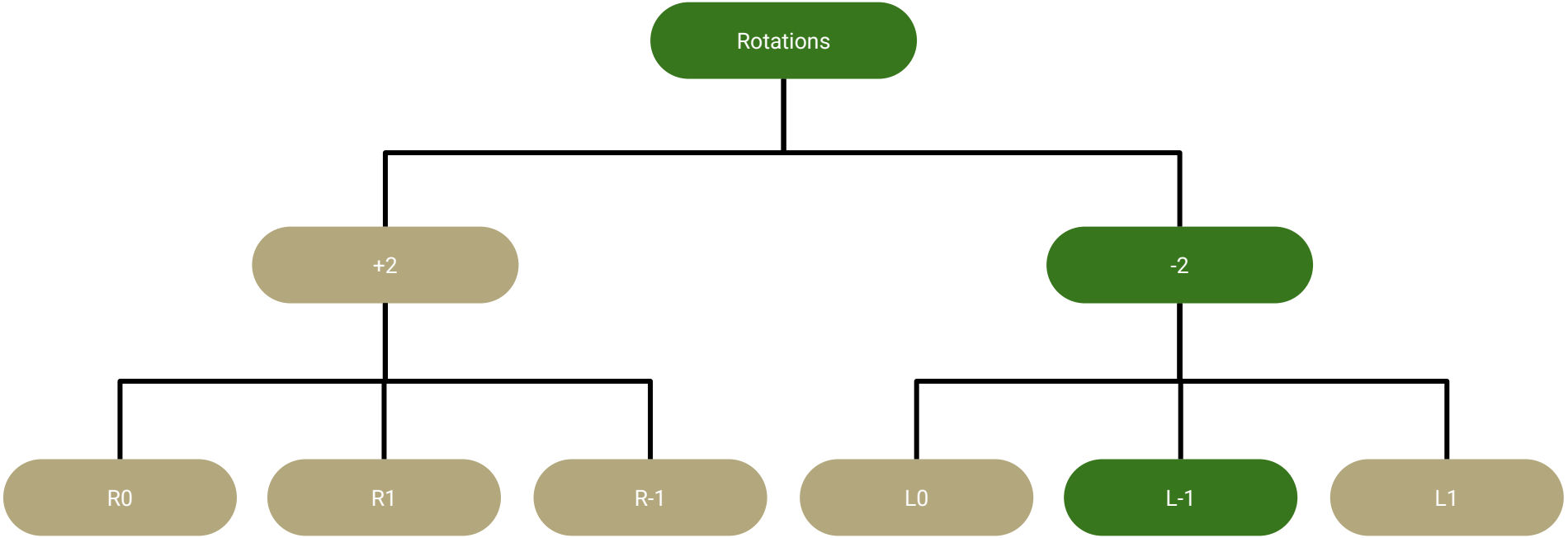
b. UNBALANCED AVL

L0 Rotation



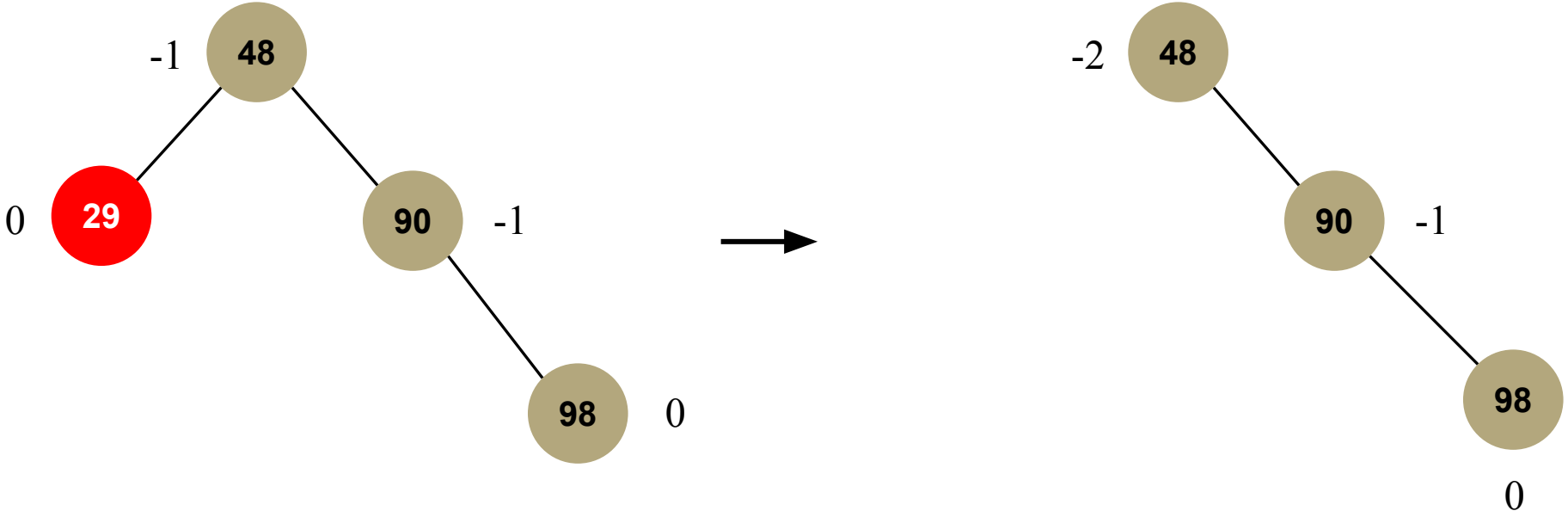
b. UNBALANCED AVL

AVL Tree Deletion



L-1 Rotation

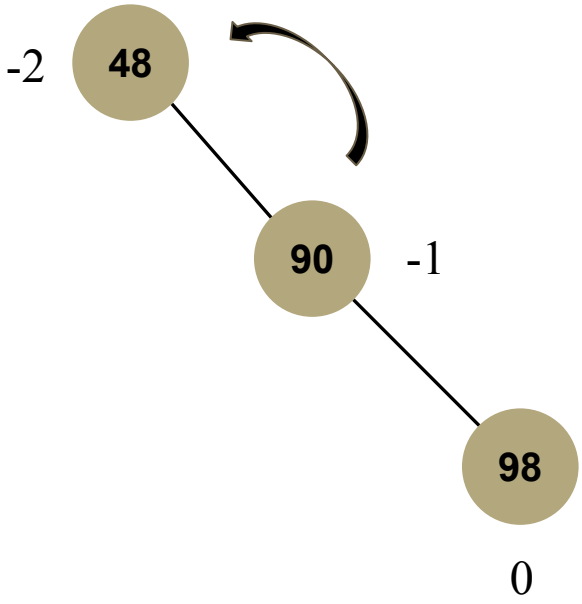
- Deletion occurred in the left subtree of A → BF(A) = -2 and (BF(B) = -1).
- Requires single left rotation around node A.

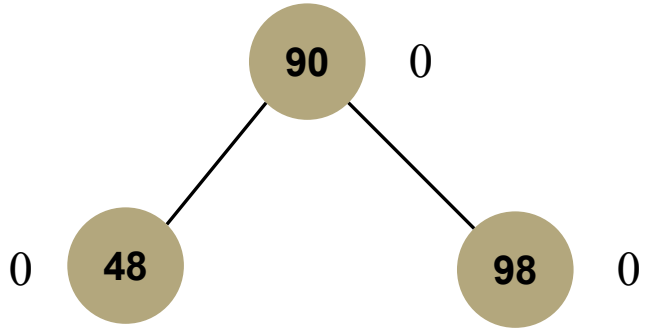


a. DELETE 29

b. UNBALANCED AVL

L-1 Rotation

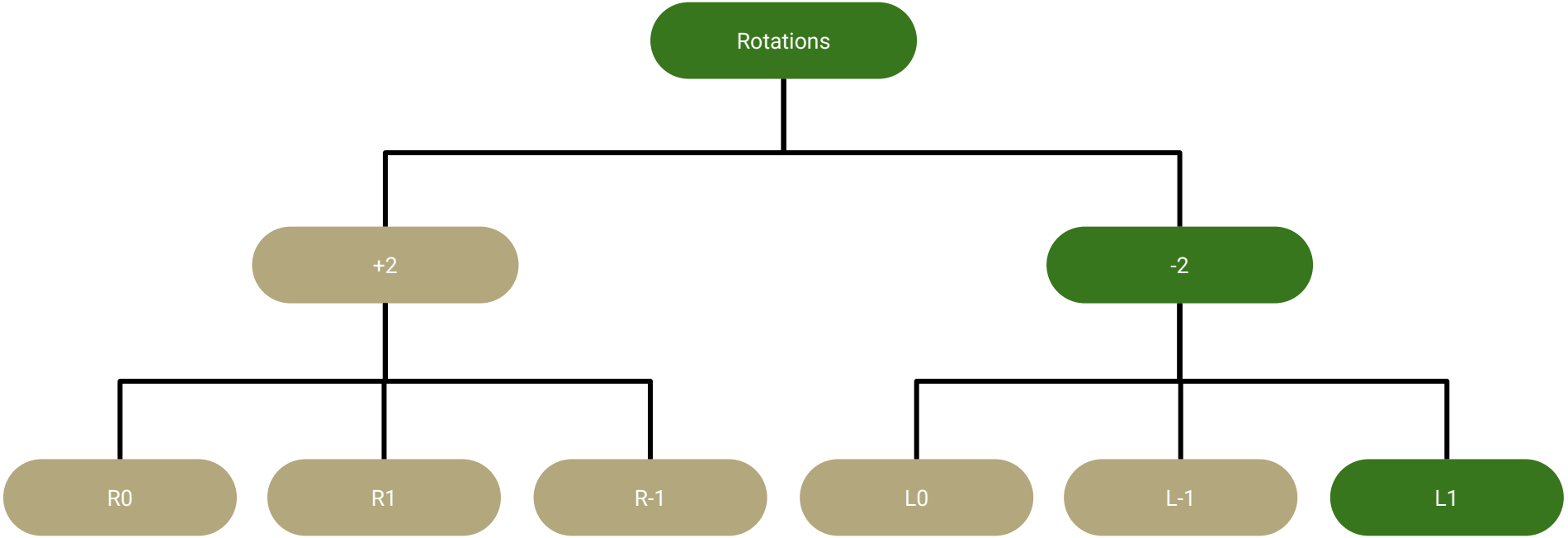




b. UNBALANCED AVL

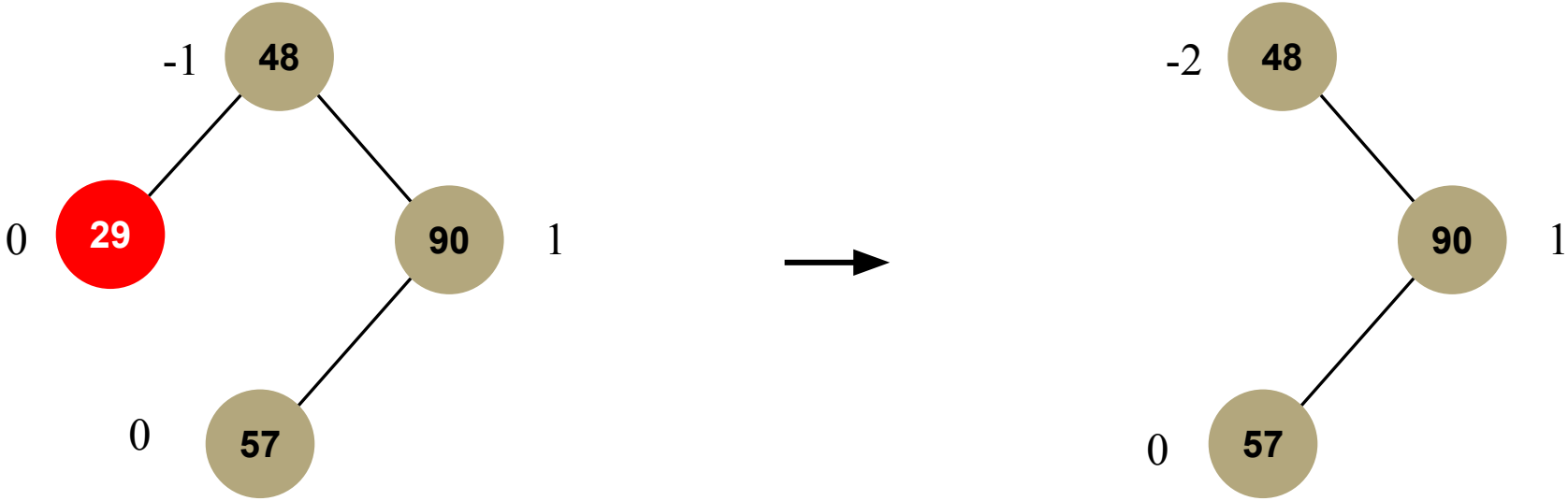
b. UNBALANCED AVL

AVL Tree Deletion



L1 Rotation

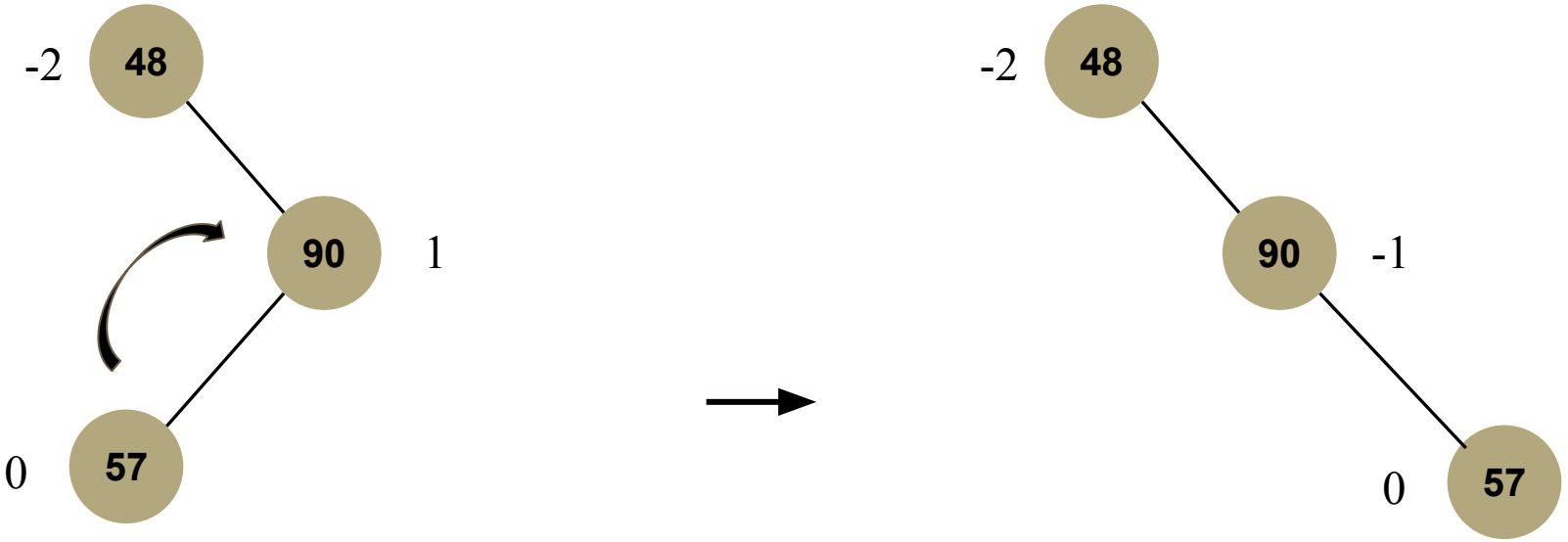
- Deletion occurred in the left subtree of A → BF(A) = -2 and (BF(B) = 1).
- Requires first right rotation on B, then left rotation on A.



a. DELETE 29

b. UNBALANCED AVL

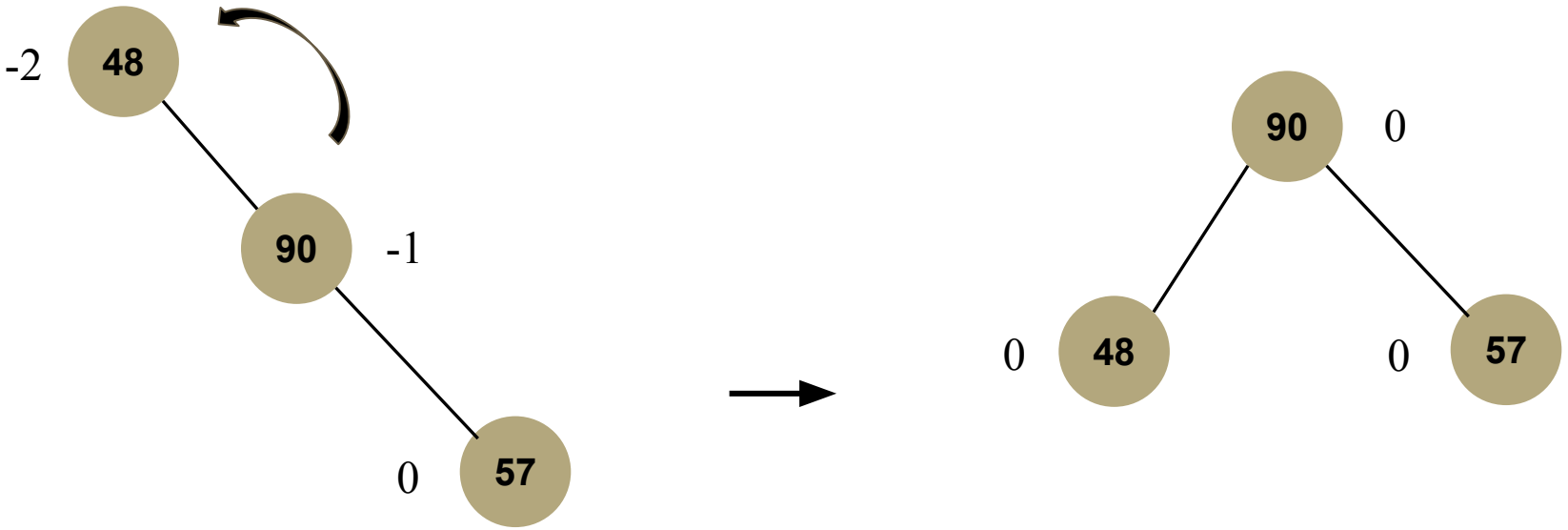
L1 Rotation



b. UNBALANCED AVL

c. UNBALANCED AVL

L1 Rotation



b. UNBALANCED AVL

c. BALANCED AVL

AVL Tree Deletion Complexity

Since an AVL tree is always height-balanced, the height remains O(log n)

Rotation operations = O(1)

Height update & balance factor calculation = O(1)

Traversal (to find node + retrace) = O(log n)

- **Best Case:** O(log n)
- **Average Case:** O(log n)
- **Worst Case:** O(log n)